

Adaptively Robust LLM Inference Optimization under Prediction Uncertainty

Zixi Chen

Department of Mathematics, Peking University, chenzixi22@stu.pku.edu.cn

Yinyu Ye

Department of Management Science and Engineering, Stanford University & Department of Industrial Engineering and
Decision Analytics, HKUST, yyeye@stanford.edu

Zijie Zhou

Department of Industrial Engineering and Decision Analytics, HKUST, jerryzhou@ust.hk

We study the problem of optimizing Large Language Model (LLM) inference scheduling to minimize total latency. LLM inference is an online and multi-task service process and also heavily energy consuming by which a pre-trained LLM processes input requests and generates output tokens sequentially. Therefore, it is vital to improve its scheduling efficiency and reduce the power consumption while a great amount of prompt requests are arriving. A key challenge in LLM inference scheduling is that while the prompt length is known upon arrival, the output length, which critically impacts memory usage and processing time, is unknown. To address this uncertainty, we propose algorithms that leverage **machine learning to predict output lengths**, assuming the prediction provides an interval classification (min-max range) for each request.

We first design a conservative algorithm, $\mathcal{A}_{\max}$, which schedules requests based on the upper bound of predicted output lengths to prevent memory overflow. However, this approach is overly conservative: as prediction accuracy decreases, performance degrades significantly due to potential overestimation. To overcome this limitation, we propose $\mathcal{A}_{\min}$, an adaptive algorithm that initially treats the predicted lower bound as the output length and dynamically refines this estimate during inferencing. We prove that $\mathcal{A}_{\min}$ achieves a log-scale competitive ratio. Through numerical simulations, we demonstrate that $\mathcal{A}_{\min}$ often performs nearly as well as the hindsight scheduler, highlighting both its efficiency and robustness in practical scenarios. Moreover, $\mathcal{A}_{\min}$ relies solely on the lower bound of the prediction interval—an advantageous design choice since upper bounds on output length are typically more challenging to predict accurately.

Key words: Robust and Online Scheduling, LLM Inference, Operations Management in AI with Prediction

1. Introduction

Recent advances in large language models (LLMs) (Brown et al. 2020, Chowdhery et al. 2023, OpenAI 2023, Kaplan et al. 2020, Wei et al. 2022) have redefined the boundaries of artificial intelligence, showcasing exceptional proficiency in generating human-like text across a wide spectrum of languages and contexts. These advanced neural architectures, built upon vast and diverse textual datasets, have become integral to a wide range of practical applications. Their influence extends to AI-driven conversational interfaces (Anthropic 2023, Character 2021, OpenAI 2019, 2023), next-generation search technologies (Microsoft 2023, Google 2023, Perplexity 2022), intelligent programming assistants (Amazon 2023, GitHub 2021), and operations service management (Huang et al. 2025, Cascella et al. 2023, Sallam 2023), highlighting their adaptability across both specialized and everyday use cases.

Large Language Model (LLM) inference—the process by which a pre-trained LLM processes input requests and generates output tokens sequentially—is not just a computational task but a critical operational challenge with real-world cost and energy implications. Optimizing this process can significantly reduce infrastructure expenses and power consumption, especially given the scale of modern LLM deployments. This problem sits at the intersection of AI systems and operations research (OR), where classical scheduling techniques, adapted to LLM-specific constraints, can unlock substantial efficiency gains.

For each request, the inference consists of two key phases:

1. *Input (Prefill) Phase:* The LLM reads the input prompt, computes the query, key, and value representations for each token in the prompt, and produces the first output token.
2. *Output (Decode) Phase:* The LLM generates subsequent tokens autoregressively—using all previously generated tokens as context—until completion.

For example, given the input prompt “KMeans is used”, the LLM might first generate the token “for”, then use “KMeans is used for” to produce “clustering”, and so on. Crucially, while the input length is known upfront, the output length is inherently unknown until generation finishes. Recent work has explored methods to predict output lengths in advance, often employing auxiliary machine learning models or even LLMs themselves (Zheng et al. (2023b), Qiu et al. (2024), Fu et al. (2024), Chen et al. (2024), Shahout et al. (2024)). Most of these approaches typically classify requests into predefined output-length intervals based on their predictions.

Given the high volume of requests we typically handle, simultaneous processing is often unfeasible. To optimize efficiency, it is essential to design scheduling algorithms using operations research techniques to prioritize request sequences. Accurate output length prediction is critical for efficient

LLM inference scheduling, especially when handling large volumes of requests. Such predictions enable effective scheduling policies by providing two key insights: (i) **Execution Time Estimation:** Since tokens are generated sequentially, the output length directly correlates with the request’s completion time. (ii) **Memory Usage Forecasting:** The key-value (KV) cache memory grows linearly during decoding, as each new token’s KV embeddings are stored and retained for all subsequent steps. Thus, predicting output length allows systems to anticipate both the computational duration and memory footprint of each request.

The design of efficient scheduling policies for LLM inference—which determines the processing order of a great number of input prompts under output length predictions—has been explored under idealized assumptions. For instance, when predictions are assumed to be perfectly accurate, prior work Jaillet et al. (2025), Shahout et al. (2024) proposes shortest-first algorithms that prioritize requests with smaller predicted output lengths. This approach offers two advantages: (i) shorter jobs finish faster, reducing the total waiting time; (ii) the system can process more requests concurrently in larger batches at early stage since shorter requests consume less KV cache memory. However, real-world predictions are inherently imperfect. Inaccurate predictions introduce two key challenges:

- **Suboptimal Scheduling:** Misclassify a long output as short disrupts the processing order, delaying other requests.
- **Memory Overflows and Underflows:** Underestimated memory demands can exceed GPU KV cache limits, forcing request cancellations and halting execution. Overestimated memory demands can reduce the concurrency, increasing the total waiting time.

This paper rigorously addresses these challenges by developing robust scheduling algorithms grounded in mathematical theory, ensuring efficiency even under prediction uncertainty.

1.1. Main Contribution

We introduce the main contribution and the outline in this section.

Naive Conservative Algorithm $\mathcal{A}_{\max}$ and its Competitive Analysis. In Section 3, we propose a naive benchmark algorithm, $\mathcal{A}_{\max}$, that conservatively treats the predicted upper bound of each job’s output length as its true length. This approach guarantees no memory overflows by over-provisioning KV cache resources for all requests. However, by overestimating the memory usage for each request, the algorithm hurts the concurrency if the prediction error is large. Our analysis reveals two key results: (i) We construct an adversarial arrival sequence proving that $\mathcal{A}_{\max}$ achieves a competitive ratio of at least $\frac{\alpha^{-1}(1+\alpha^{-1/2})}{2}$, where α is the ratio of the prediction interval’s lower bound to its upper bound. (ii) Moreover, we rigorously show the upper bound

of the competitive ratio of $\mathcal{A}_{\max}$ is $\frac{\alpha^{-1}(1+\alpha^{-1})}{2}$. The proof leverages a novel memory-preserving combinatorial technique (Section EC.1), which may inspire further research in LLM inference scheduling under prediction uncertainty.

Robust Advanced Algorithm $\mathcal{A}_{\min}$ and its Competitive Analysis. Motivated by the observation that the competitive ratio of $\mathcal{A}_{\max}$ becomes unbounded as α shrinks (i.e., when predictions lack confidence), we design $\mathcal{A}_{\min}$, a robust algorithm that leverages only the lower prediction bound. Initially, $\mathcal{A}_{\min}$ underestimates each request’s output length as its lower bound, dynamically refining this estimate during execution. In Section 4, we prove that $\mathcal{A}_{\min}$ achieves asymptotic optimality with a competitive ratio of $\mathcal{O}(\log(\alpha^{-1}))$, demonstrating significantly stronger robustness than $\mathcal{A}_{\max}$. The analysis in Subsection 4.1 is notably more challenging than that of $\mathcal{A}_{\max}$: we derive the competitive ratio’s closed form as a Rayleigh quotient and then develop a novel estimation technique to bound this quotient logarithmically. In Section 5, we also study the performance of $\mathcal{A}_{\min}$ over some specific output distributions. We show that under two-point distribution, geometric distribution, linear weighted geometric distribution, the competitive ratios of $\mathcal{A}_{\min}$ is uniformly bounded by 1.5, 1.7, and 1.56 respectively.

Numerical Experiments. In Section 6, we evaluate our algorithms through three sets of experiments on a real-world dataset Zheng et al. (2023a). First, we establish a baseline where all requests share an identical prediction interval, representing perfect prediction uniformity. Second, we implement a binned configuration where requests are categorized into ten discrete prediction intervals based on their predictions. Third, we evaluate a fully individualized setting where each request is assigned its own unique prediction interval centered around the true output length, better reflecting practical scenarios where prediction confidence varies per request. Notably, while the second experiment groups requests by interval similarity, the third preserves each request’s distinct interval characteristics. Across all three configurations, $\mathcal{A}_{\max}$ demonstrates strong performance only when predictions are highly accurate, while $\mathcal{A}_{\min}$ proves remarkably robust, consistently matching or approaching the performance of the hindsight algorithm that operates with perfect knowledge of the true output lengths.

1.2. Other Related Work

Optimization in LLM Inference. While existing LLM inference literature has predominantly focused on system design and engineering optimizations Patel et al. (2023), Zhong et al. (2024), Yu et al. (2022), Agrawal et al. (2023, 2024b), recent work has begun establishing theoretical foundations for LLM inference scheduling. The seminal work of Jaillet et al. (2025) first formalized a

mathematical framework for analyzing algorithm performance in this context. Our work extends this foundation by introducing prediction intervals—where algorithms must operate robustly without knowledge of true output lengths. Parallel developments include Ao et al. (2025), which incorporates multi-metric scheduling constraints, Wang et al. (2025), which designs efficient scheduling algorithms for the setting where the input size is heterogeneous, and Li et al. (2025), which characterizes stability conditions for LLM inference systems. Together, these works mark the emergence of theoretical optimization approaches complementing the field’s traditional systems focus.

Decision-Making and Operations Management with Prediction Uncertainty. Our work connects to broader research on learning-augmented algorithms, where predictions inform decision-making under uncertainty. While Shahout et al. (2024) develops prediction methods for LLM inference without considering robust scheduling, other domains have deeply explored this paradigm: Lykouris and Vassilvitskii (2021) analyzes online caching with access predictions, Golrezaei et al. (2023) examines online resource allocation with predicted demand, Agrawal et al. (2011) studies the prediction market design, and Jin and Ma (2022), Antoniadis et al. (2020) investigates prediction-enhanced secretary problem and online matching. The robust optimization literature (Bertsimas and Sim 2004) also provides foundational techniques for handling uncertainty, while recent advances by Zhou et al. (2022) and Zhang et al. (2023) demonstrate applications to scheduling problems. Our work bridges these perspectives by developing robust policies specifically for prediction-informed LLM scheduling frameworks.

Online and Offline Scheduling. The operations research community has extensively studied online and offline scheduling problems (Chen et al. 1998, Blazewicz et al. 2007, Mak et al. 2015, Leung et al. 2010). These problems involve determining the processing order and start times for a large set of jobs that cannot be executed simultaneously. Prior work includes batch scheduling, where jobs are processed in parallel or grouped into batches (Xing and Zhang 2000, Yang et al. 2003, Cao et al. 2005, Chen and Lee 2008), and resource-constrained scheduling, where jobs consume limited resources during execution (Brucker et al. 1999, Hiermann et al. 2015).

The LLM inference scheduling problem studied in our work combines these two dimensions: it is both batch-based and resource-constrained, with GPU memory acting as a reusable resource. Crucially, the memory usage patterns of LLM inference exhibit unique characteristics that distinguish our problem from classical scheduling models. These distinct features necessitate the development of new algorithmic approaches tailored to this setting.

2. Problem Settings

In this paper, we study an extension of the LLM inference scheduling model proposed in Jaillet et al. (2025). We begin by reviewing the mathematical framework presented in Jaillet et al. (2025) and then describe the modifications introduced in our model. We consider a setting with a single computational worker equipped with a KV cache of size $M > 0$, capable of storing up to M tokens. In practice, the value of M depends on the complexity of the large language model and the available hardware memory of the computational worker. We assume that M is known to the decision-maker. A total of n jobs (prompts) are waiting in the queue, where each job i has a size $s_i > 0$, representing the number of tokens in the prompt. Following the assumptions in Jaillet et al. (2025), and motivated by the observation that in real-world parallel computing environments prompts assigned to a worker typically have similar lengths, we assume $s_i = s > 0$ for all $i \in [n]$. The value of s is known to the decision-maker.

Prediction Interval for Output Length. To process each prompt, the model generates the output token by token. Let $o_i > 0$ denote the realized output length of job i , representing the number of tokens generated in its response. Since the realized value o_i is not revealed until the last output token is generated, it remains unknown to the decision-maker during the process. In Jaillet et al. (2025), it is assumed that the realized output length o_i is perfectly predicted and known in advance. In contrast, our current model assumes that o_i is unknown a priori but can be predicted to lie within an interval $[\ell, u]$, and we denote $\alpha = \frac{\ell}{u}$. For clarity and to build intuition regarding the complexity of the problem and the design of our algorithm, we first consider the case where all jobs share the same predicted interval $[\ell, u]$. We then generalize the result in Section 4.2 to allow each job’s output length to fall within different predicted intervals.

Batch Processing, Memory Constraint, and Cancellation of Requests. Following the model in Jaillet et al. (2025), we allow jobs to be processed in batches. At each discrete time step t , the scheduler selects a subset of jobs $S^{(t)}$ to form a batch for processing. Each batch takes exactly one unit of time to process. For any job $i \in S^{(t)}$, let $a_i^{(t)} \in \{0, 1, \dots, o_i\}$ denote the number of output tokens it has generated by time t . If $a_i^{(t)} < o_i$, then processing the batch at time t results in the generation of the $(a_i^{(t)} + 1)$ -th token for job i . Once $a_i^{(t)} = o_i$, job i is fully completed.

Each batch must also satisfy a memory constraint determined by the KV cache limit M . At any time t , let $\mathcal{A}^{(t)}$ denote the set of active jobs—i.e., those that have begun processing in some earlier batch. For each active job $i \in \mathcal{A}^{(t)}$, the memory required is $s + a_i^{(t)}$, accounting for both the prompt size and the number of output tokens generated so far. The total memory usage at time t must not exceed the limit M , yielding the following constraint:

$$\sum_{i \in \mathcal{A}^{(t)}} (s + a_i^{(t)}) \leq M, \quad \forall t \geq 0. \quad (1)$$

Since our setting assumes that only interval predictions of o_i are available, it is hard to guarantee feasibility of constraint (1) under some non-preemptive policies. For example, suppose the total memory usage at time t is $M - 1$ with two active jobs, each with at least two tokens remaining. Even if the decision-maker delays one job and processes the other, both jobs will eventually require at least 2 units of additional memory. Thus, there is no way to complete both without exceeding the memory constraint in some future time step.

To address this challenge, based on the non-preemptive structure, our model permits the cancellation of jobs. That is, the scheduler may cancel any active job i at time t , discarding all previously generated output tokens $a_i^{(t)}$ and resetting the job’s state to unprocessed. We emphasize that since the sampling method is greedy, the total realized output length o_i of each job remains fixed, regardless of how many times it is cancelled and restarted.

Evaluation Metrics. We use total end-to-end latency as our performance metric. Denote the vector $\mathbf{o} = (o_1, o_2, \dots, o_n)$ which contains the true value of output length of all requests. W.L.O.G, we assume that $o_1 \leq o_2 \leq \dots \leq o_n$. Under a scheduling policy π , all prompts are arranged into an input queue $I = (I_0, I_1, \dots, I_T)$, where I_t denotes the set of prompts whose last (and final) starting time is at time t —that is, they are not cancelled after t . For each request $i \in I_t$, we define its latency as $L_i = t + o_i$, which corresponds to the completion time of its last output token. Since all jobs arrive at time $t = 0$, this is also the total end-to-end latency of request i , and the total end-to-end latency of the system is then given by

$$\text{TEL}(\mathbf{o}; \pi) := \sum_{i \in [n]} L_i = \sum_{t=0}^T t \cdot |I_t| + \sum_{i \in [n]} o_i. \quad (2)$$

The second equality follows by grouping all jobs according to their final starting time: each job in I_t contributes latency t from waiting and o_i from processing. The first term sums all waiting times, while the second term sums all output lengths.

3. Benchmark Algorithms

In this section, we introduce several benchmark algorithms to evaluate a given scheduling policy π . A central challenge in our model is that the decision-maker does not know the exact output length o_i of each request i in advance. Instead, the only information available is that $o_i \in [\ell, u]$, and we define the uncertainty parameter as $\alpha = \frac{\ell}{u}$. The true value of each o_i is selected adversarially from this interval at the outset. To assess performance under this uncertainty, Subsection 3.1 introduces a hindsight benchmark algorithm from Jaillet et al. (2025), in which all output lengths o_i are known ahead of time. We also define the competitive ratio as a metric for evaluating policies that operate without access to the true values of o_i . In Subsection 3.2, we propose a naive benchmark algorithm that lacks knowledge of o_i , and analyze its competitive ratio.

3.1. Review of Hindsight Benchmark Algorithm and Competitive Ratio

In the hindsight setting—where the decision-maker has complete knowledge of the output lengths o_i for all $i \in [n]$ —our model closely aligns with the setting in Jaillet et al. (2025), with one key distinction: we allow for job cancellations. However, when the exact values of o_i are known in advance, cancellations are unnecessary. This is because the memory usage over time can be precisely anticipated and managed using the approach introduced in Jaillet et al. (2025) as follows: at each time t , let R_t be the set of pending requests awaiting processing. Suppose we consider adding a subset $U \subset R_t$ to the batch. Define $t_{\max}(U) := \max_{i \in U} \{t + o_i\}$, which represents the latest time any job in U will complete if processing starts at time t . To ensure that the memory constraint is respected throughout this interval, we must verify that the total memory consumption remains below the KV cache limit M at every time $t' \in [t, t_{\max}(U)]$. This requirement is formalized by the following constraint:

$$\sum_{i \in S^{(t)}} (s + t' - p_i) \cdot \mathbb{1}_{\{o_i \geq t' - p_i\}} + \sum_{i \in U} (s + t' - t) \cdot \mathbb{1}_{\{o_i \geq t' - t\}} \leq M, \quad \forall t' \in [t, t_{\max}(U)], \quad (3)$$

where $S^{(t)}$ denotes the set of jobs already in progress at time t , and p_i is the last start time of job i . The first summation captures memory usage from ongoing jobs, while the second accounts for the new jobs in U . As long as this constraint is satisfied for all relevant time points, the batch is guaranteed to respect the memory limit and no cancellations are needed.

Furthermore, when the decision-maker has full knowledge of the values o_i , either in theory (Jaillet et al. (2025)) or in practical implementations (Shahout et al. (2024), Zheng et al. (2023b), Qiu et al. (2024), Fu et al. (2024), Chen et al. (2024)), one effective batching strategy is shortest-job-first. The classical OR heuristic attempts to include as many of the shortest remaining jobs with respect to o_i as possible in each batch without violating constraint (3). The rationale is twofold: first, shorter jobs contribute less to waiting time, reducing overall latency; second, since the peak memory usage of a job is $s + o_i$, shorter jobs also occupy less memory, allowing more requests to be packed into earlier batches. We refer to this hindsight benchmark as (Hindsight-Shortest First) H-SF, and describe it formally in Appendix EC.2. Next, we describe the definition of competitive ratio:

DEFINITION 1 (COMPETITIVE RATIO). Let $\mathbf{o} = (o_1, o_2, \dots, o_n) \in [\ell, u]^n$ denote the vector of true output lengths for all requests. For a scheduling policy π , let $TEL(\mathbf{o}; \pi)$ denote the total end-to-end latency under policy π , and let $TEL(\mathbf{o}; \text{H-SF})$ denote the latency achieved by H-SF which has full knowledge of $\mathbf{o}$. Then, the competitive ratio of policy π is defined as:

$$CR(\pi) := \sup_{\mathbf{o} \in [\ell, u]^n} \frac{\mathbb{E}[TEL(\mathbf{o}; \pi)]}{TEL(\mathbf{o}; \text{H-SF})}.$$

3.2. Naive Benchmark Algorithm

Now consider the setting where the decision-maker only has access to the prediction interval $[\ell, u]$ for each job. Since all intervals are identical, the decision-maker cannot distinguish between jobs and possesses no information to prioritize one over another. It can only reason about the range within which each job’s output length may fall.

To handle this uncertainty, we introduce a naive benchmark algorithm, Max-Length Based Algorithm, denoted by $\mathcal{A}_{\max}$. This algorithm assumes the worst-case scenario by treating every job as if its output length equals u . At each time step, it selects the maximum number of prompts that can be processed without violating the memory constraint in Equation (3). Since the output lengths are overestimated, the algorithm guarantees that the memory limit is never exceeded, and thus no cancellations are required. The detail information can be found in Algorithm 1.

Algorithm 1: $\mathcal{A}_{\max}$

Input: Memory Capacity M , prompt size s , output length lower and bound $\ell \leq o_i \leq u$ for all $i \in [n]$.

Output: Processing sequence $I = (I_0, I_1, \dots, I_T)$.

while *there exist waiting requests* **do**

Let R_t be the set of waiting prompts at time t . Let S_t be the set of tokens currently processing at time t .

Take all $o_i = u$, find the largest cardinality value m_t such that there exists a set $U \subset R_t$ such that $|U| = m_t$ and Equation (3) hold for all $t' \geq t$.

Randomly sample $I_t \subset R_t$ with $|I_t| = m_t$.

Process the requests in $I_t \cup S_t$ and update $R_{t+1} = R_t \setminus I_t$.

end

Although $\mathcal{A}_{\max}$ guarantees feasibility, it may suffer from inefficiency due to its overly conservative estimation of output lengths. By assuming each job has the maximum possible length u , it overestimates the peak memory usage per job and may significantly under-utilize the available KV cache memory. The following example illustrates this inefficiency:

EXAMPLE 1. Consider a setting with $n = 5$ requests, each with input size $s = 1$, and output lengths $o_i \in [1, 4]$. Suppose the true values are $o_i = 1$ for all $i \in [5]$, and the memory capacity is $M = 10$.

The hindsight-optimal algorithm $H\text{-SF}$, having access to the true output lengths, knows that the peak memory usage per job is $s + o_i = 2$. Hence, it can batch all 5 jobs together, fully utilizing the

memory $M = 2 \times 5 = 10$. Each job completes after 1 unit of processing time, resulting in a latency of 1 per job and total end-to-end latency of $5 \times 1 = 5$.

In contrast, $\mathcal{A}_{\max}$ assumes $o_i = 4$ for all jobs, leading to a peak memory estimate of $s + u = 5$ per job. It therefore batches only $\lfloor M/5 \rfloor = 2$ jobs at a time. It first processes 2 jobs, then the next 2, and finally the last remaining job. The resulting job completion times are 1, 1, 2, 2, and 3, yielding a total latency of $1 + 1 + 2 + 2 + 3 = 9$.

This example highlights the inefficiency of $\mathcal{A}_{\max}$, particularly in cases where the prediction interval is wide. As the gap between ℓ and u increases—i.e., as $\alpha = \ell/u$ decreases—the degree of memory under-utilization becomes more severe. The following theorem establishes an upper bound on the competitive ratio of $\mathcal{A}_{\max}$.

THEOREM 1. *The competitive ratio of $\mathcal{A}_{\max}$ is upper bounded by:*

$$CR(\mathcal{A}_{\max}) \leq \frac{\alpha^{-1}(1 + \alpha^{-1})}{2} + \mathcal{O}\left(\frac{1}{M}\right).$$

While the example above provides intuition for where the inefficiency of $\mathcal{A}_{\max}$ arises, formally analyzing its competitive ratio is extremely challenging. The difficulty stems from the complex dynamics of the model, particularly the fact that each job’s memory usage increases linearly over time during processing. Therefore, it is hard to find the closed-form connection between the total end-to-end latency and the memory constraint. To address this challenge, we introduce a combinatorial structure that captures the underlying memory evolution in a concise and analyzable way. This *memory-preserving* structure serves as a key proof technique and offers a general framework for analyzing scheduling problems under similar memory constraints. We use a separate section to make the full proof, and one can find the proof in Appendix EC.1.

While Theorem 1 provides an upper bound of competitive ratio of $\mathcal{A}_{\max}$, this ratio is not tight because it treats all output lengths as equal to u , which causes a mismatch between the effect of output lengths on latency and the effect of the input order. As a result, our analysis cannot fully capture how the actual input sequence influences the scheduling performance, leading to a loose upper bound. Then, we also provide a lower bound of competitive ratio of $\mathcal{A}_{\max}$.

THEOREM 2. *The competitive ratio of $\mathcal{A}_{\max}$ is lower bounded by*

$$CR(\mathcal{A}_{\max}) \geq \frac{\alpha^{-1}(1 + \alpha^{-1/2})}{2}.$$

Proof of Theorem 2 Take $s = 0$, recall $\frac{\ell}{u} = \alpha$. Let $M = \ell u$, and the algorithm $\mathcal{A}_{\max}$ is assigned with $N_\ell = au$ prompts with output length ℓ and $N_u = b\ell$ prompts with output length u . Here, we take $\frac{a}{b} \approx \alpha^{1/2}$ and a, b are large enough. Because $\mathcal{A}_{\max}$ regards all prompts as being of size u , it will only initiate a new request if there is memory u available. This is equivalent to considering it as a combination of $\frac{M}{u} = \ell$ parallel workers, where any one of them can only process one prompt at a time and start a new one randomly after finishing a prompt.

Then it is simple to calculate $\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\max})] = (au + b\ell)\frac{au\ell + b\ell u}{2\ell}$, since all prompts have the expected output time $\frac{au\ell + b\ell u}{2\ell}$. Also, $\text{TEL}(\mathbf{o}; \mathbf{H}\text{-SF}) = u\ell(1 + \dots + a) + b\ell a\ell + u\ell(1 + \dots + b)$.

We get

$$\text{CR}(\mathcal{A}_{\max}) \geq \frac{u}{\ell} \frac{(au + b\ell)(a + b)}{ua^2 + \ell 2ab + ub^2} = \frac{\alpha^{-1}(1 + \alpha^{-1/2})}{2}.$$

□

4. Robust Advanced Scheduling Algorithm

Previously, we introduced a naive scheduling algorithm, $\mathcal{A}_{\max}$, and analyzed its competitive ratio, which is asymptotically between $\frac{\alpha^{-1}(1+\alpha^{-1/2})}{2}$ and $\frac{\alpha^{-1}(1+\alpha^{-1})}{2}$ according to Theorems 1 and 2, where $\alpha = \frac{\ell}{u}$ captures the prediction accuracy of the output lengths o_i . This result shows that the performance of $\mathcal{A}_{\max}$ is highly sensitive to the quality of the prediction model. In the ideal case where the prediction is perfect (i.e., $\ell = u$, so $\alpha = 1$), the competitive ratio equals to 1, indicating optimal performance. However, in practice, decision-makers often prioritize speed over accuracy in prediction. When the prediction interval is wide (i.e., $\alpha \rightarrow 0$), the competitive ratio grows unbounded with the rate between $\mathcal{O}(\alpha^{-2})$ and $\mathcal{O}(\alpha^{-1.5})$, demonstrating that $\mathcal{A}_{\max}$ is highly non-robust. This observation raises a key question: can we design a scheduling algorithm that performs well not only under accurate predictions, but also maintains a better competitive ratio under poor prediction quality? In this section, we introduce an advanced and robust policy denoted $\mathcal{A}_{\min}$.

Recall that in $\mathcal{A}_{\max}$, each request is pessimistically assumed to have the maximum output length u . When the true output length o_i is much smaller than u , this conservative estimate leads to significant memory overprotection, reducing the number of simultaneously processed requests and increasing latency. To address this, we approach the problem from the opposite perspective. If the goal is to maximize the number of requests in each batch, it is natural to use the lower bound ℓ as a proxy for each request's output length. Unlike the upper bound u , the lower bound can be dynamically updated: initially, we know only that $o_i \geq \ell$, we create a variable $\tilde{o}_i$ for each request i as its output length lower bound and initialize $\tilde{o}_i = \ell$ at beginning. But once a request has generated $\tilde{\ell}_i$ tokens, we know $o_i \geq \tilde{\ell}_i$, and we can update the lower bound accordingly, namely $\tilde{o}_i = \tilde{\ell}_i$. Based on this idea, our advanced algorithm $\mathcal{A}_{\min}$ proceeds in two key steps:

1. *Memory Overflow Resolution via Ordered Eviction.* Because the true output length o_i is not known in advance and may exceed $\tilde{o}_i$, the algorithm may encounter a situation where continuing to process the current batch would exceed the memory limit. In this case, $\mathcal{A}_{\min}$ removes jobs from the batch to bring the memory usage back within the constraint. Specifically, it orders the currently active jobs by increasing $\tilde{o}_i$, breaks ties randomly, and removes jobs one by one in this order until the projected memory usage becomes feasible. This strategy ensures that the algorithm prefers to retain requests with larger accumulated lower bounds, which are more expensive to restart. Moreover, for each canceled request i , we update the value of $\tilde{o}_i$ to reflect the number of output tokens already generated, which is a lower bound of the true output length o_i .

2. *Batch Formation Based on Sorted Lower Bounds.* At time $t = 0$, the algorithm sets $\tilde{o}_i = \ell$ for all requests i . To form a batch, it sorts the current set of unprocessed or restartable requests in ascending order of $\tilde{o}_i$, breaking ties uniformly at random. It then greedily selects as many requests as possible from the sorted list—starting from the smallest $\tilde{o}_i$ —until the memory constraint is reached.

Importantly, since the sampling process is greedy and deterministic, removing a request i does not change its true output length o_i . Thus, even if a request i is removed and later restarted, the algorithm retains its current lower bound $\tilde{o}_i$ at the time of removal. This updated value is used for future batch formation. The auxiliary lower bound $\tilde{o}_i$ is updated only when a job i is cancelled, since a job continues processing uninterrupted once selected into a batch; thus, there is no need to update $\tilde{o}_i$ while the job remains active. Upon cancellation, however, the number of output tokens already generated provides a new certified lower bound on the true output length o_i , and this updated $\tilde{o}_i$ is used to inform future batch decisions. This strategy avoids unnecessary computation while preserving the algorithm’s adaptiveness and memory efficiency.

Key Design Advantage: No Need for Upper Bound Prediction. An important feature of $\mathcal{A}_{\min}$ is that it operates solely using the predicted lower bounds ℓ , without ever relying on the predicted upper bounds u . This makes the algorithm significantly more practical and robust: in many real-world settings, generating reliable lower bounds is much easier and faster than estimating sharp upper bounds. By avoiding dependence on u , $\mathcal{A}_{\min}$ remains effective even when the prediction intervals are highly uncertain or asymmetric, making it more suitable for deployment in systems where inference-time estimates must be generated quickly or under weak supervision. The formal pseudocode for $\mathcal{A}_{\min}$ is presented in Algorithm 2.

Next, we start to analyze the theoretical performance of $\mathcal{A}_{\min}$. First, the following theorem states the computational complexity of $\mathcal{A}_{\min}$ is polynomial, and the proof can be found in Appendix EC.4.

Algorithm 2: $\mathcal{A}_{\min}$

Input: Memory capacity M , prompt size s , **output length lower bounds $o_i \geq \ell$** for all $i \in [n]$.

Output: Processing sequence $I = (I_0, I_1, \dots, I_T)$.

Initialize $\tilde{o}_i \leftarrow \ell$ for all $i \in [n]$.

while *there exist unfinished requests* **do**

 Let R_t be the set of waiting prompts at time t . Let S_t be the set of tokens currently processing at time t .

if *projected memory usage of S_t at time $t+1$ exceeds M* **then**

 Sort S_t in ascending order of $\tilde{o}_i$; break ties uniformly at random.

 Remove jobs one by one from S_t (following the sorted order) until projected memory usage at $t+1$ satisfies the memory constraint.

 For each removed request i , update $\tilde{o}_i \leftarrow$ number of tokens already generated by request i .

end

 Let S'_t be the set of remaining active requests after any removals.

 Sort R_t in ascending order of $\tilde{o}_i$; break ties uniformly at random.

 Select the largest subset $I_t \subset R_t$ (in order) such that adding I_t to S'_t satisfies the memory constraint in Equation (3).

 Process the requests in $I_t \cup S'_t$.

 Update $R_{t+1} = R_t \setminus I_t$.

end

THEOREM 3. *Consider the KV cache memory limit is M , Algorithm $\mathcal{A}_{\min}$ has a computational complexity of $\mathcal{O}(M \log M)$.*

Next, Theorem 4 states that no algorithm can achieve a better performance than $\mathcal{A}_{\min}$ when $M \rightarrow \infty$, showing the asymptotically optimality. The proof can also be found in Appendix EC.4.

THEOREM 4. *For any feasible policy π without the full information of $\mathbf{o} = (o_1, o_2, \dots, o_n)$, when $M \rightarrow \infty$, the competitive ratio is lower bounded by*

$$CR(\pi) \geq CR(\mathcal{A}_{\min}).$$

Although Theorem 4 proves the optimality of $\mathcal{A}_{\min}$, it is still necessary to analyze its competitive ratio to quantify its improvement over $\mathcal{A}_{\max}$. However, like $\mathcal{A}_{\max}$, deriving an upper bound on this ratio is highly challenging. The next subsection presents our proof approach and theoretical results.

4.1. Logarithm Competitive Ratio Bound of $\mathcal{A}_{\min}$

In this subsection, our main purpose is to introduce our novel method to prove the following theorem:

THEOREM 5. *While $M \rightarrow \infty$, the asymptotical competitive ratio of $\mathcal{A}_{\min}$ is $\mathcal{O}(\log(\alpha^{-1}))$.*

To prove Theorem 5, we divide the analysis into two parts. In Part 1, we derive a closed-form expression for the competitive ratio of $\mathcal{A}_{\min}$. In Part 2, we rigorously bound this expression and show that it scales logarithmically.

Part 1: Get a Closed-form expression for the competitive ratio.

We begin with two key observations:

- *Observation 1:* The execution of $\mathcal{A}_{\min}$ can be partitioned into several periods: $\tau_\ell, \tau_{\ell+1}, \dots, \tau_u$. Before the start of the period τ_i , all requests q with real output length $o_q < i$ have already been completed. In period τ_i , $\mathcal{A}_{\min}$ attempts to initialize all requests with a lower bound on output length $\tilde{o} = i$. For example, in the first period, all requests are loaded into memory and processed until they are either completed or deleted. If a request q is deleted, its associated lower bound is updated to a new value $\tilde{o}_q > i$. During τ_i , we denote by b_{ij} , c_{ij} , and d_{ij} the numbers of input, completed, and deleted requests with the true output length $o = j$, respectively. These quantities satisfy the following relationship.

LEMMA 1. *We have*

$$\mathbb{E}[c_{ij}|b_{ij}] = \frac{s+i}{s+j} b_{ij}, \quad \mathbb{E}[d_{ij}|b_{ij}] = \left(1 - \frac{s+i}{s+j}\right) b_{ij}.$$

The proof of Lemma 1 can be found in Appendix EC.4.

- *Observation 2:* In period τ_i , if a request q is deleted and reassigned a new lower bound $\tilde{o}_q = k$, its true output length must satisfy $o_q \geq \tilde{o}$. Again, by input symmetry, the probability that its true output length is $o = j \geq k$ is proportional to b_{ij} . This is because $\mathcal{A}_{\min}$ breaks ties uniformly when selecting inputs. Formally, we have

$$\mathbb{P}_i(o = j \mid \tilde{o} = k) = \frac{b_{ij}}{\sum_{t \geq k} b_{it}}. \quad (4)$$

For a fixed request set $\mathbf{o}_n$ of n requests, where x_i is the proportion of the number of requests with output length i in the request set $\mathbf{o}_n$, the above two observations allow us to derive the expected state transitions across all periods.

PROPOSITION 1. *Given the request set $\mathbf{o}_n$, we have*

$$\mathbb{E}[c_{ij}] = \begin{cases} \frac{s+i}{s+j}x_jn, & \text{if } i = \ell, \\ \frac{1}{s+j}x_jn, & \text{otherwise.} \end{cases} \quad (5)$$

The proof of Proposition 1 can be found in Appendix EC.4. Proposition 1 indicates that in the initial period τ_ℓ , $\mathcal{A}_{\min}$ successfully processes more requests with shorter lengths. Moreover, the proportion of completed requests decreases with j according to the factor $\frac{s+\ell}{s+j}$. In the subsequent periods, the expected number of completed requests remains constant at $\frac{1}{s+j}$ times x_jn . Although this expected completion ratio does not change across periods, $\mathcal{A}_{\min}$ achieves it by continually updating the lower bound $\tilde{o}$ in earlier periods. This demonstrates that $\mathcal{A}_{\min}$ can effectively defer the input of longer requests by actively learning and utilizing structural information from previous steps. We can now derive the competitive ratio of $\mathcal{A}_{\min}$.

LEMMA 2. *Let*

$$\delta_{s,l}(k) = \begin{cases} s+l, & \text{if } k = l \\ 1, & \text{if } k \neq l \end{cases}$$

We have

$$CR(\mathcal{A}_{\min}) = \frac{\sum_{k=\ell}^u \delta_{s,l}(k) (\sum_{i=k}^u ix_i) (\sum_{i=k}^u \frac{\delta_{s,l}(k)}{s+i} x_i + 2 \sum_{i=k+1}^u \frac{i-k}{s+i} x_i)}{\sum_{k=\ell}^u k(s+k)x_k(x_k + 2 \sum_{i=k+1}^u x_i)} + \mathcal{O}(\frac{1}{M}).$$

Proof of Lemma 2 We compute the two components of

$$\frac{\mathbb{E}[\text{TEL}(\mathbf{o}_n; \mathcal{A}_{\min})]}{\text{TEL}(\mathbf{o}_n; \text{H-SF})}.$$

For the hindsight algorithm, which processes requests starting from the shortest ones, the total processing time for requests of output length k is $\frac{k(s+k)x_kn}{M}$. At this point, requests with length k are symmetric, and the average latency increase for these requests is $\frac{k(s+k)x_kn}{2M}$. Additionally, the remaining $\sum_{i=k+1}^u x_i n$ requests must wait, incurring a further latency of $\frac{k(s+k)x_kn}{M}$.

Summing over all k from ℓ to u , we obtain

$$\text{TEL}(\mathbf{o}_n; \text{H-SF}) = \frac{n^2}{2M} \left(\sum_{k=\ell}^u k(s+k)x_k \left(x_k + 2 \sum_{i=k+1}^u x_i \right) \right) (1 + \mathcal{O}(\frac{1}{M})).$$

For the algorithm $\mathcal{A}_{\min}$, it successfully completes a portion of the requests in each stage, leading to an expected time cost of

$$\mathbb{E}[\tau_k] = \sum_{i=k}^u \frac{i(s+i)}{M} \mathbb{E}[c_{ii}] = \frac{\delta_{s,l}(k)}{M} \sum_{i=k}^u ix_i n.$$

Furthermore, the expected number of remaining requests is $\sum_{i=k+1}^u \frac{i-k}{s+i} x_i n$.

Summing over k from ℓ to u , we get

$$\mathbb{E}[\text{TEL}(\mathbf{o}_n; \mathcal{A}_{\min})] = \frac{n^2}{2M} \left(\sum_{k=\ell}^u \delta_{s,l}(k) \left(\sum_{i=k}^u i x_i \right) \left(\sum_{i=k}^u \frac{\delta_{s,l}(k)}{s+i} x_i + 2 \sum_{i=k+1}^u \frac{i-k}{s+i} x_i \right) \right) (1 + \mathcal{O}(\frac{1}{M})).$$

Dividing by $\text{TEL}(\mathbf{o}_n; \mathbf{H}\text{-SF})$ and taking the limit as $n \rightarrow \infty$, we conclude that

$$\text{CR}(\mathcal{A}_{\min}) = \frac{\sum_{k=\ell}^u \delta_{s,l}(k) (\sum_{i=k}^u i x_i) (\sum_{i=k}^u \frac{\delta_{s,l}(k)}{s+i} x_i + 2 \sum_{i=k+1}^u \frac{i-k}{s+i} x_i)}{\sum_{k=\ell}^u k(s+k) x_k (x_k + 2 \sum_{i=k+1}^u x_i)} + \mathcal{O}(\frac{1}{M}).$$

□

To see the asymptotic performance of $\text{CR}(\mathcal{A}_{\min})$, W.L.O.G., we take $s = 0$, $\ell = 1$, then, $\alpha = \frac{\ell}{u} = \frac{1}{u}$.

The next corollary writes the competitive ratio of $\mathcal{A}_{\min}$ in the matrix form:

COROLLARY 1. *Let $\vec{x} = (x_1, \dots, x_u)^\top$, and define $\mathbf{A}_u = (a_{ij})_{u \times u}$ and $\mathbf{B}_u = (b_{ij})_{u \times u}$, where*

$$a_{ij} = \frac{\min(i, j)^2}{ij} \left(\frac{1}{2} (i+j)^2 - \min(i, j)^2 \right),$$

$$b_{ij} = \min(i, j)^2.$$

The competitive ratio of $\mathcal{A}_{\min}$ simplifies to the following Rayleigh quotient,

$$\text{CR}(\mathcal{A}_{\min}) = \max_{\vec{x}} \frac{\vec{x}^\top \mathbf{A}_u \vec{x}}{\vec{x}^\top \mathbf{B}_u \vec{x}} + \mathcal{O}(\frac{1}{M}).$$

The proof of Corollary 1 can be found in Appendix EC.4. Our goal is to show that the Rayleigh quotient

$$R(\vec{x}) := \frac{\vec{x}^\top \mathbf{A}_u \vec{x}}{\vec{x}^\top \mathbf{B}_u \vec{x}}$$

is uniformly bounded by $\mathcal{O}(\log u)$ for all $\vec{x} > 0$.

Part 2: Bound the Rayleigh quotient.

To provide the upper bound of $R(\vec{x})$, we start by showing that the two matrices $\mathbf{A}_u$ and $\mathbf{B}_u$ are both positive definite.

LEMMA 3 (Positive Definiteness of $\mathbf{A}_u$ and $\mathbf{B}_u$). *For all integers $u \geq 1$, both $\mathbf{A}_u$ and $\mathbf{B}_u$ are positive definite.*

The proof of Lemma 3 can be found in Appendix EC.4.

Consider the matrix

$$\mathbf{C}_u := \mathbf{B}_u^{-1} \mathbf{A}_u.$$

Since both $\mathbf{A}_u$ and $\mathbf{B}_u$ are symmetric and positive definite (by Lemma 3), the matrix $\mathbf{C}_u$ is similar to a symmetric positive definite matrix, and thus has real, strictly positive eigenvalues. In particular, the maximum of the Rayleigh quotient satisfies

$$\max_{\vec{x} > 0} R(\vec{x}) \leq \max_{\vec{x} \in \mathbb{R}^u} R(\vec{x}) = \rho(\mathbf{C}_u),$$

where $\rho(\mathbf{C}_u)$ denotes the spectral radius, i.e., the largest eigenvalue of $\mathbf{C}_u$.

Since all eigenvalues of $\mathbf{C}_u$ are positive, its spectral radius is bounded above by the sum of its eigenvalues. Therefore, it suffices to estimate the trace:

$$\rho(\mathbf{C}_u) \leq \text{tr}(\mathbf{C}_u) = \text{tr}(\mathbf{B}_u^{-1} \mathbf{A}_u).$$

We now compute the trace of $\mathbf{C}_u = \mathbf{B}_u^{-1} \mathbf{A}_u$ explicitly using the known expressions for $\mathbf{A}_u$ and the inverse of $\mathbf{B}_u$. Recall that $\mathbf{B}_u^{-1}$ is tridiagonal, with entries

$$(\mathbf{B}_u^{-1})_{ij} = \begin{cases} \frac{4i}{4i^2 - 1}, & i = j < u, \\ \frac{1}{2u - 1}, & i = j = u, \\ -\frac{1}{2\min(i, j) + 1}, & |i - j| = 1, \\ 0, & \text{otherwise.} \end{cases}$$

Using the symmetry of $\mathbf{A}_u$, the trace can be written as the sum of products over all entries of $\mathbf{B}_u^{-1}$ and $\mathbf{A}_u$ along the main and immediate subdiagonals:

$$\text{tr}(\mathbf{B}_u^{-1} \mathbf{A}_u) = \sum_{i=1}^u (\mathbf{B}_u^{-1})_{ii} a_{ii} + 2 \sum_{i=1}^{u-1} (\mathbf{B}_u^{-1})_{i,i+1} a_{i,i+1}.$$

We define the two contributions as:

$$\begin{aligned} \text{(diagonal terms), } T_1(u) &:= \sum_{i=1}^u (\mathbf{B}_u^{-1})_{ii} a_{ii} = \sum_{i=1}^{u-1} \frac{4i}{4i^2 - 1} i^2 + \frac{1}{2u - 1} u^2, \\ \text{(subdiagonal terms), } T_2(u) &:= \sum_{i=1}^{u-1} (\mathbf{B}_u^{-1})_{i,i+1} a_{i,i+1} = - \sum_{i=1}^{u-1} \frac{1}{2i + 1} \frac{i}{i + 1} (i^2 + 2i + \frac{1}{2}), \end{aligned}$$

so that

$$\text{tr}(\mathbf{B}_u^{-1} \mathbf{A}_u) = T_1(u) + 2T_2(u).$$

Explicit computation yields the following closed-form expressions:

- For the diagonal term:

$$T_1(u) = \frac{u^2 + 1}{2} + \frac{1}{2} \sum_{i=1}^{u-1} \frac{1}{2i + 1}.$$

- For the subdiagonal term:

$$2T_2(u) = -\frac{u^2 - 1}{2} + \sum_{i=1}^{u-1} \frac{1}{i+1} - \frac{1}{2} \sum_{i=1}^{u-1} \frac{1}{2i+1}.$$

Combining all terms:

$$\text{tr}(\mathbf{B}_u^{-1} \mathbf{A}_u) = T_1(u) + 2T_2(u) = \sum_{i=1}^u \frac{1}{i} = \mathcal{O}(\log u).$$

Therefore, the spectral radius of $\mathbf{C}_u$ is also $\mathcal{O}(\log u)$, and the Rayleigh quotient satisfies:

$$\max_{\vec{x} > 0} \frac{\vec{x}^\top \mathbf{A}_u \vec{x}}{\vec{x}^\top \mathbf{B}_u \vec{x}} = \mathcal{O}(\log u),$$

which implies that the competitive ratio of the algorithm $\mathcal{A}_{\min}$ satisfies:

$$\text{CR}(\mathcal{A}_{\min}; \mathcal{D}) = \mathcal{O}(\log u) = \mathcal{O}(\log(\alpha^{-1})).$$

□

We now have analyzed the Rayleigh quotient theoretically, establishing a logarithmic competitive ratio for $\mathcal{A}_{\min}$. Finally, we numerically estimate the Rayleigh quotient for the case $s = 0$ and $\ell = 1$ to verify the theoretical result. As shown in Figure 1, the logarithmic function provides an excellent fit, with an R^2 value exceeding 0.9999.

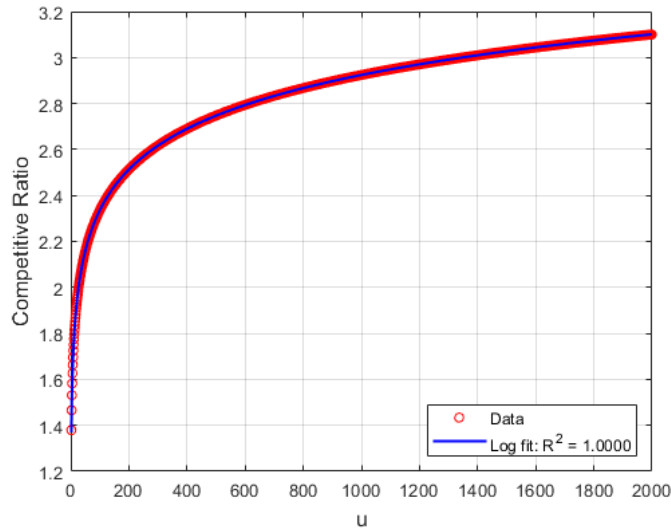


Figure 1 Competitive ratio of $\mathcal{A}_{\min}$ versus upper bound u , fitted by $0.2555 \log(u + 0.2793) + 1.1587$, assuming $s = 0, \ell = 1$.

4.2. Heterogeneous Prediction Intervals

Now, we extend our framework by replacing the single prediction interval with m disjoint intervals: $[\ell_1, u_1], [\ell_2, u_2], \dots, [\ell_m, u_m]$, satisfying $u_j \leq \ell_{j+1}$. This formulation naturally models classification-based prediction approaches, where a machine learning model first predicts an output length for each request and then assigns it to the corresponding interval.

Our algorithm, $\mathcal{A}_{\min}$, naturally extends to this generalized setting with minimal modifications. For each request i classified into interval $[\ell_j, u_j]$, we initialize its lower bound as ℓ_j rather than using a uniform initial value across all requests. This modified version (Algorithm 5 in Appendix EC.4) guarantees that the initial underestimation becomes more precise for each individual request, and the algorithm consequently achieves better overall performance. As formalized in the following corollary, this adaptation yields a competitive ratio at least equals to the basic setting. We validate the numerical performance in Experiment 2 in Section 6, demonstrating the practical benefits of incorporating refined prediction intervals.

COROLLARY 2. *Under prediction intervals $[\ell_1, u_1], [\ell_2, u_2], \dots, [\ell_m, u_m]$, satisfying $u_j \leq \ell_{j+1}$, take $\alpha = \frac{\ell_1}{u_m}$. While $M \rightarrow \infty$, the asymptotical competitive ratio of the revised $\mathcal{A}_{\min}$ is $\mathcal{O}(\log(\alpha^{-1}))$.*

5. Extensions: $\mathcal{A}_{\min}$ and New Algorithms under Specific Output Distributions

In previous sections, we assumed the adversary selects request output values within the prediction interval $[\ell, u]$. Here, we examine another variant where outputs follow some specific distributions $\mathcal{D}$ over $[\ell, u]$, and analyze $\mathcal{A}_{\min}$'s performance under this setting. Notably, $\mathcal{A}_{\min}$ operates solely using ℓ without distributional knowledge. This motivates our investigation of new algorithms that can leverage $\mathcal{D}$ to improve performance.

We focus on three representative distributions:

1. Two-point distribution: All outputs are either ℓ or u , representing an extreme case;
2. Geometric distribution: Outputs exhibit exponential decay, a pattern commonly observed in real-world datasets.
3. Linearly weighted Geometric distribution: Combines exponential decay with a linearly weighted preference for values near the lower bound ℓ . This modified distribution better captures real-world phenomena where extreme values occur more frequently than standard geometric decay would predict.

5.1. Two-Point Distribution

In this section, we examine a special case where the output length $o \in \{\ell, u\}$. We denote this two-point distribution family as $\mathcal{D}_2$. We start by evaluating the theoretical performance of $\mathcal{A}_{\min}$.

THEOREM 6. *Under any distribution $\mathcal{D} \in \mathcal{D}_2$, the competitive ratio of $\mathcal{A}_{\min}$ is bounded by:*

$$CR(\mathcal{A}_{\min}) \leq \frac{3-\alpha}{2} + \mathcal{O}\left(\frac{1}{M}\right).$$

The complete proof appears in Appendix EC.5. Theorem 6 establishes that for any $\alpha \in [0, 1]$, the competitive ratio of $\mathcal{A}_{\min}$ is bounded above by $\frac{3}{2}$. This represents a significant improvement over $\mathcal{A}_{\max}$, whose competitive ratio is unbounded in the worst case – a scenario that occurs precisely within the two-point distribution family $\mathcal{D}_2$. Our result thus demonstrates that $\mathcal{A}_{\min}$ achieves bounded competitiveness where $\mathcal{A}_{\max}$ fails.

Next, we introduce the *promote- ℓ policy* $\mathcal{A}_\ell$, which uses the information that the output distribution belongs to $\mathcal{D}_2$. Under this policy, the server processes each incoming request sequentially. As soon as the currently served request generates ℓ tokens, the policy infers that the true length of this request must be $u > \ell$. At that point, $\mathcal{A}_\ell$ removes the request, appends it to the end of the queue, and proceeds to the next one. Any request of length $\leq \ell$ is completed normally. In effect, $\mathcal{A}_\ell$ defers all long jobs after ℓ tokens, postponing them until all shorter jobs are finished. The detailed algorithm can be found in Algorithm 6 in Appendix EC.5. Next, we provide the theoretical competitive analysis of $\mathcal{A}_\ell$.

THEOREM 7. *Under any distribution $\mathcal{D} \in \mathcal{D}_2$, the competitive ratio of $\mathcal{A}_\ell$ is bounded by:*

$$CR(\mathcal{A}_\ell) \leq \begin{cases} 1 + \frac{\alpha}{2(1-\alpha)}, & 0 < \alpha \leq \frac{1}{2}, \\ 1 + 2\alpha^2, & \frac{1}{2} \leq \alpha \leq 1, \end{cases} + \mathcal{O}\left(\frac{1}{M}\right), \quad \text{where } \alpha = \ell/u.$$

The proof of Theorem 7 can be found in Appendix EC.5. Next, based on the results in Theorems 6 and 7, we can study when to choose $\mathcal{A}_{\min}$ and $\mathcal{A}_\ell$. From Theorem 6, we know that $CR(\mathcal{A}_{\min}) \leq (3-\alpha)/2 + \mathcal{O}(1/M)$, while Theorem 7 gives a piecewise upper bound for $CR(\mathcal{A}_\ell)$. To compare the two when $0 \leq \alpha \leq \frac{1}{2}$, we consider:

$$CR(\mathcal{A}_\ell) < CR(\mathcal{A}_{\min}) \iff 1 + \frac{\alpha}{2(1-\alpha)} < \frac{3-\alpha}{2},$$

which reduces to the quadratic inequality $\alpha^2 - 3\alpha + 1 = 0$. Solving yields the critical threshold:

$$\alpha^* = \frac{3 - \sqrt{5}}{2} \approx 0.382.$$

Since $\alpha = \ell/u$, the condition $\alpha < \alpha^*$ translates to $u \gtrsim 2.62\ell$; that is, *when the long request is more than 2.6 times larger than the short one, $\mathcal{A}_\ell$ achieves a smaller competitive ratio*. For $\alpha \geq \alpha^*$, the monotonic bound $1 + 2\alpha^2$ dominates, making $\mathcal{A}_{\min}$ the preferable choice.

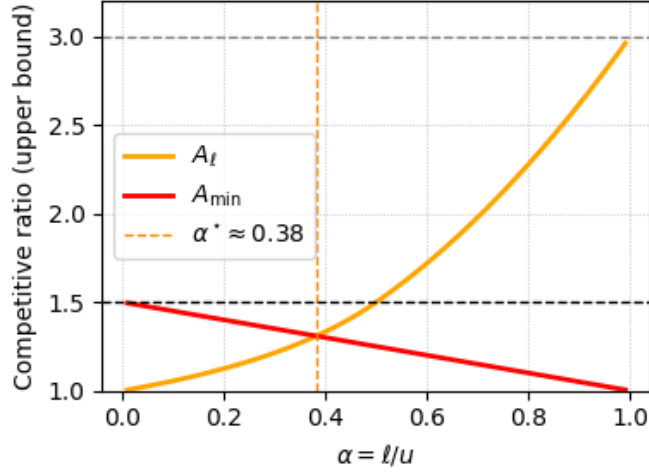


Figure 2 Upper-bound curves for the competitive ratios under $\mathcal{D}_2$. When $\alpha < \alpha^* \approx 0.38$ (left of the dashed line), $\mathcal{A}_\ell$ outperforms $\mathcal{A}_{\min}$; beyond that point, the inequality reverses.

Practical takeaway. Rather than committing to a single scheduling policy, one can *adaptively* choose between $\mathcal{A}_{\min}$ and $\mathcal{A}_\ell$ based on the empirical ratio $\alpha = \ell/u$ observed in the workload. This simple switching strategy ensures the smaller of the two analytical bounds shown in Figure 2, improving the worst-case guarantee from 1.5 to $1 + \frac{\alpha}{2(1-\alpha)}$ in high-skew scenarios, while never exceeding the red curve in low-skew regimes. More broadly, this illustrates a general principle: taking advantage of distributional features, such as the length ratio α , to select among multiple algorithms, can significantly enhance the robustness of competitive-ratio guarantees.

5.2. Geometric Distribution

Let $G(p)$ denote the geometric distribution with parameter p , such that $o \sim G(p)$ and $\mathbb{P}(o = k) = pq^{k-1}$, where $q = 1 - p$. The $G(p)$ describes an exponential decay of o on the discrete space $[1, u]$. Under the geometric distribution, it is hard to provide a theoretical result for the competitive ratio. However, the left plot in Figure 3 indicates that the competitive ratio of $\mathcal{A}_{\min}$ increases when q increases, and is bounded by 1.7.

5.3. Linearly Weighted Geometric Distribution

Let $LG(p)$ denote the linearly weighted geometric distribution with the parameter p , $o \sim LG(p)$ and $\mathbb{P}(o = k) = kp^2q^{k-1}$, where $q = 1 - p$. Compared to $G(p)$, the linearly weighted geometric distribution more accurately reflects practical scenarios, as it exhibits increasing mass for small values of o and reaches its peak around $\log \frac{1}{q}$.

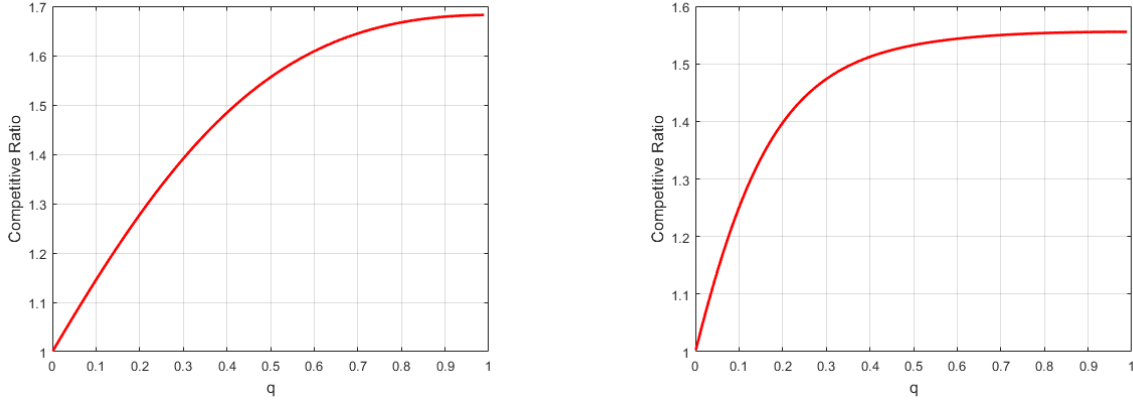


Figure 3 Competitive ratio as a function of parameter q : (Left) geometric distribution $G(p)$; (Right) linearly weighted geometric distribution $LG(p)$.

The right plot of Figure 3 shows that the competitive ratio of $\mathcal{A}_{\min}$ again increases when q increases, and is bounded by 1.6. Furthermore, a closed-form expression can be derived for the linearly weighted geometric distribution $LG(p)$, and the tight competitive ratio upper bound is about 1.56. The following theorem shows the result and the proof can be found in Appendix EC.5.

THEOREM 8. *As $u \rightarrow \infty$ and $M \rightarrow \infty$, the competitive ratio of $\mathcal{A}_{\min}$ under any linearly weighted geometric distribution $\mathcal{D} \in LG(p)$ is given by*

$$CR(\mathcal{A}_{\min}) = \frac{(1+q)^2(1+3q+6q^2+3q^3+q^4)}{1+2q+11q^2+8q^3+11q^4+2q^5+q^6} \leq \frac{14}{9} \approx 1.56.$$

6. Numerical Experiments

In this section, we evaluate the performance of different scheduling algorithms using a real-world dataset under various assumptions about the accuracy of output length predictions.

Dataset Overview. We use the LMSYS-Chat-1M dataset released by Zheng et al. (2023a), available at <https://huggingface.co/datasets/lmsys/lmsys-chat-1m>, which consists of conversations collected from over 210,000 unique IP addresses via the Vicuna demo and the Chatbot Arena platform. For our numerical experiments, we randomly sample a subset of 2,000 conversations.

For each conversation, we define the input size as the number of tokens in the prompt and the output length as the number of tokens in the corresponding model response. Among the selected 2,000 conversations, the input sizes range from 1 to 468 tokens, with a mean of 41, median of 11, and variance of 4,961. The output lengths range from 1 to 883 tokens, with a mean of 85, median

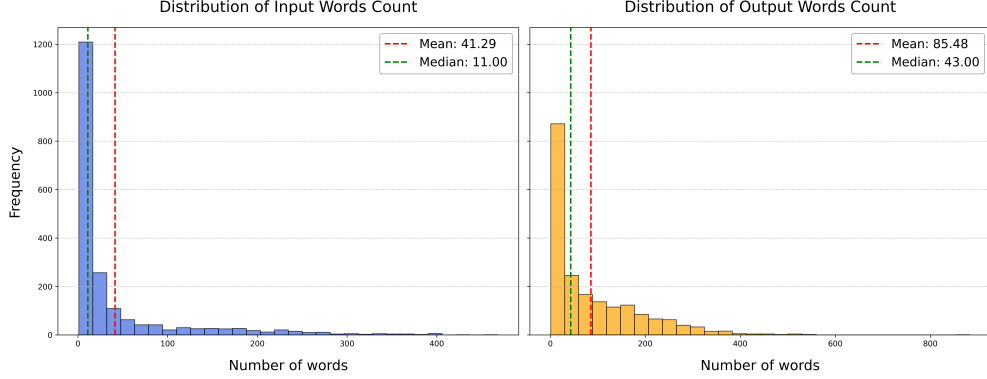


Figure 4 Distribution of the number of words of input prompt and output response respectively

of 43, and variance of 9,702. Figure 4 displays the empirical distributions of input sizes and output lengths.

Experiment Setup. We evaluate the scheduling algorithms under three prediction settings that reflect varying levels of output length accuracy:

1. **Rough Prediction:** Each request is assumed to have a coarse prediction interval of $[1, 1000]$, representing minimal information about the output length.
2. **Non-Overlapping Classification:** Based on the true output length of each request, we assign its prediction interval to one of the fixed buckets: $[1, 100], [101, 200], \dots, [901, 1000]$, simulating a multi-class classification model with non-overlapping intervals.
3. **Overlapping Interval Prediction:** For each request i with true output length o_i , the prediction interval is set as $[(1 - x)o_i, (1 + x)o_i]$, where $x \in (0, 1)$ controls the accuracy of the prediction. Larger values of x correspond to more precise predictions.

To observe latency trends under different load levels, we vary the number of requests in each simulation, selecting random subsets of size 200, 400, ..., up to 2,000 from the original dataset. We report the average end-to-end latency, defined as the total latency divided by the number of requests.

In all experiments, we compare the average latency of $\mathcal{A}_{\max}$ and $\mathcal{A}_{\min}$. As a benchmark, we include the H-SF algorithm, which assumes perfect knowledge of each request’s output length and serves as a lower bound on achievable latency. Batch processing time is estimated using the linear regression predictor in Vidur simulator from Agrawal et al. (2024a) under the simulation environment of the LLaMA2-70B model on two linked A100 GPUs.

Results for Experiment 1. In Experiment 1, the prediction interval for all requests is set to the broad range $[1, 1000]$. Under this setting, $\mathcal{A}_{\max}$ pessimistically assumes that each request has an

output length of 1000. While this conservative approach avoids memory overflows, it leads to poor memory utilization—each batch includes very few requests, resulting in high latency. In contrast, $\mathcal{A}_{\min}$ initializes the lower bound of each request as 1 and adaptively updates this value during the inference process as tokens are generated.

Figure 5 shows that $\mathcal{A}_{\min}$ achieves average latency nearly identical to the benchmark H-SF, which has full knowledge of the true output lengths. This result is particularly striking given that $\mathcal{A}_{\min}$ operates with minimal information—only that each request’s output lies somewhere in the interval $[1, 1000]$. It highlights the robustness and adaptiveness of $\mathcal{A}_{\min}$ even under highly uncertain predictions.

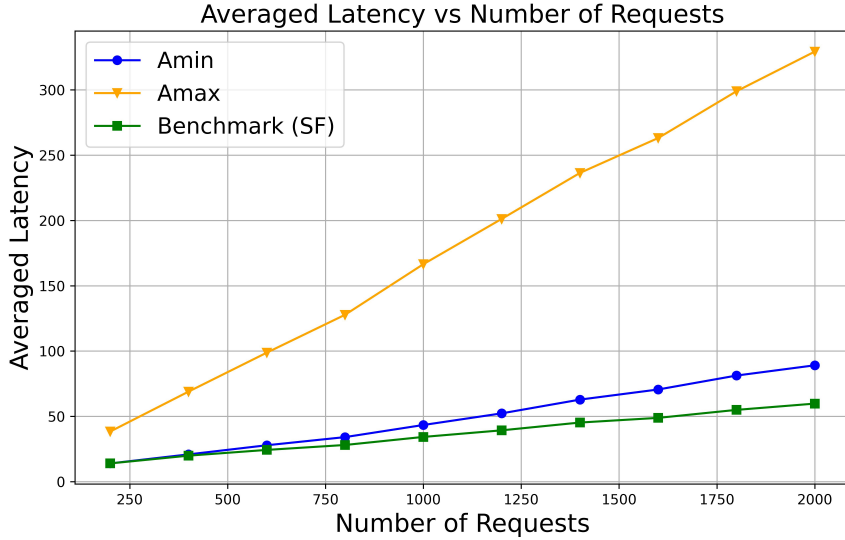


Figure 5 Averaged latency between scheduling algorithms when the prediction for every request’s output length is $[1, 1000]$.

Results for Experiment 2. In Experiment 2, we classify requests into 10 groups, each associated with a non-overlapping output length prediction interval: $[1, 100]$, $[101, 200]$, $\dots$, $[901, 1000]$. Compared to Experiment 1, this setting provides significantly more accurate predictions. Figure 6 shows the average latency of the three scheduling algorithms.

First, we observe that $\mathcal{A}_{\max}$ achieves a substantial improvement over its performance in Experiment 1—the average latency is nearly halved. This is because $\mathcal{A}_{\max}$ now treats each request as having an output length equal to the upper bound of its assigned group interval, allowing more requests to be packed into each batch compared to the uniform assumption of 1000 tokens in Experiment 1.

More importantly, Figure 6 also demonstrates that as prediction accuracy improves, the performance of $\mathcal{A}_{\min}$ closely approaches that of the benchmark H-SF. This highlights a key strength of $\mathcal{A}_{\min}$: its ability to adaptively leverage better predictions to achieve latency performance nearly identical to an ideal scheduler with full information.

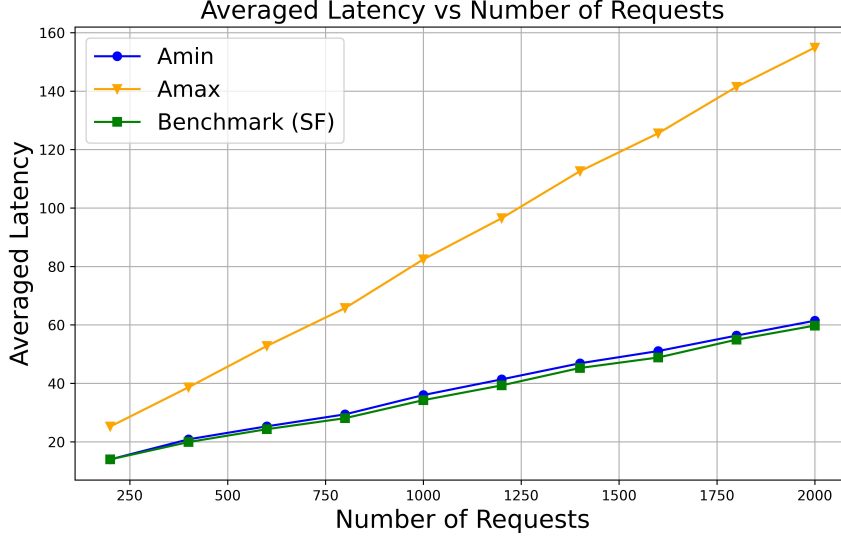


Figure 6 Averaged latency between scheduling algorithms when the prediction for every request’s output length is one of $[1, 100], [101, 200], \dots, [901, 1000]$.

Results for Experiment 3. In Experiment 3, each request i is assigned a prediction interval of the form $[(1-x)o_i, (1+x)o_i]$, where $x \in \{0.1, 0.95, 0.99\}$ controls the accuracy of the prediction. A smaller value of x corresponds to more accurate predictions.

As shown in Figure 7, when $x = 0.1$, indicating highly accurate predictions, both $\mathcal{A}_{\max}$ and $\mathcal{A}_{\min}$ perform well, achieving low average latency. However, as the prediction accuracy decreases (i.e., $x = 0.95$ or $x = 0.99$), the performance of $\mathcal{A}_{\max}$ deteriorates significantly. This is because the algorithm treats each request pessimistically by assuming its output length is near the upper bound of its interval, which leads to low memory utilization and small batch sizes.

In contrast, $\mathcal{A}_{\min}$ continues to maintain low average latency even under highly uncertain predictions. Its performance remains close to the hindsight benchmark H-SF, highlighting the robustness of $\mathcal{A}_{\min}$ in the face of imprecise output length estimates.

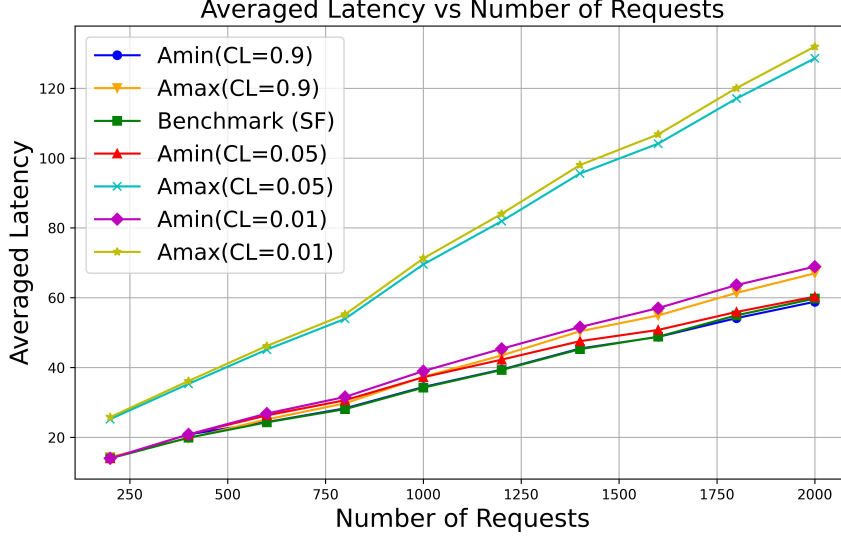


Figure 7 Averaged latency between scheduling algorithms when the prediction for every request i 's output length is $[(1-x)o_i, (1+x)o_i]$.

7. Conclusion

This paper extends the LLM inference scheduling model introduced by Jaillet et al. (2025) to a more realistic setting where only interval-based predictions of output lengths are available. We first analyze $\mathcal{A}_{\max}$, a conservative algorithm that avoids memory overflow by overestimating request lengths, but demonstrate its fundamental limitation: poor robustness when prediction intervals are wide, leading to excessive resource allocation and reduced concurrency. Our key contribution is $\mathcal{A}_{\min}$, an adaptive algorithm that employs strategic underestimation followed by progressive refinement during execution. Through both theoretical analysis and comprehensive numerical experiments, we establish that $\mathcal{A}_{\min}$ achieves robust performance across varying prediction quality while maintaining high system efficiency. These results provide practical insights for deploying LLM inference systems under prediction uncertainty.

References

- Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024a. Vidur: A Large-Scale Simulation Framework For LLM Inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024b. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. *arXiv preprint arXiv:2403.02310* (2024).
- Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).
- Shipra Agrawal, Erick Delage, Mark Peters, Zizhuo Wang, and Yinyu Ye. 2011. A unified framework for dynamic prediction market design. *Operations research* 59, 3 (2011), 550–568.
- Amazon. 2023. Amazon CodeWhisperer. <https://aws.amazon.com/codewhisperer/>.
- Anthropic. 2023. Claude. <https://claude.ai>.
- Antonios Antoniadis, Themis Gouleakis, Pieter Kleer, and Pavel Kolev. 2020. Secretary and online matching problems with machine learned advice. *Advances in Neural Information Processing Systems* 33 (2020), 7933–7944.
- Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. 2025. Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints. *arXiv preprint arXiv:2504.11320* (2025).
- Dimitris Bertsimas and Melvyn Sim. 2004. The price of robustness. *Operations research* 52, 1 (2004), 35–53.
- Jacek Blazewicz, Klaus H Ecker, Erwin Pesch, Gunter Schmidt, and Jan Weglarz. 2007. *Handbook on scheduling: from theory to applications*. Springer.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- Peter Brucker, Andreas Drexl, Rolf Möhring, Klaus Neumann, and Erwin Pesch. 1999. Resource-constrained project scheduling: Notation, classification, models, and methods. *European journal of operational research* 112, 1 (1999), 3–41.
- Dong Cao, Mingyuan Chen, and Guohua Wan. 2005. Parallel machine selection and job scheduling to minimize machine cost and job tardiness. *Computers & operations research* 32, 8 (2005), 1995–2012.
- Marco Cascella, Jonathan Montomoli, Valentina Bellini, and Elena Bignami. 2023. Evaluating the feasibility of ChatGPT in healthcare: an analysis of multiple clinical and research scenarios. *Journal of medical systems* 47, 1 (2023), 33.
- Character. 2021. Character AI. <https://character.ai>.
- Bo Chen and Chung-Yee Lee. 2008. Logistics scheduling with batching and transportation. *European journal of operational research* 189, 3 (2008), 871–876.

- Bo Chen, Chris N Potts, and Gerhard J Woeginger. 1998. A review of machine scheduling: Complexity, algorithms and approximability. *Handbook of Combinatorial Optimization: Volume 1–3* (1998), 1493–1641.
- Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. 2024. KVDirect: Distributed Disaggregated LLM Inference. *arXiv preprint arXiv:2501.14743* (2024).
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. *arXiv preprint arXiv:2408.15792* (2024).
- GitHub. 2021. GitHub Copilot. <https://github.com/features/copilot>.
- Negin Golrezaei, Patrick Jaillet, and Zijie Zhou. 2023. Online resource allocation with convex-set machine-learned advice. *arXiv preprint arXiv:2306.12282* (2023).
- Google. 2023. Bard. <https://bard.google.com>.
- Gerhard Hiermann, Matthias Prandtstetter, Andrea Rendl, Jakob Puchinger, and Günther R Raidl. 2015. Metaheuristics for solving a multimodal home-healthcare scheduling problem. *Central European Journal of Operations Research* 23, 1 (2015), 89–113.
- Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. 2025. Orlm: A customizable framework in training large models for automated optimization modeling. *Operations Research* (2025).
- Patrick Jaillet, Jiashuo Jiang, Konstantina Mellou, Marco Molinaro, Chara Podimata, and Zijie Zhou. 2025. Online Scheduling for LLM Inference with KV Cache Constraints. *arXiv preprint arXiv:2502.07115* (2025).
- Billy Jin and Will Ma. 2022. Online bipartite matching with advice: Tight robustness-consistency tradeoffs for the two-stage model. *Advances in Neural Information Processing Systems* 35 (2022), 14555–14567.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- Joseph Y-T Leung, Michael Pinedo, and Guohua Wan. 2010. Competitive two-agent scheduling and its applications. *Operations Research* 58, 2 (2010), 458–469.
- Yueying Li, Jim Dai, and Tianyi Peng. 2025. Throughput-optimal scheduling algorithms for llm inference and ai agents. *arXiv preprint arXiv:2504.07347* (2025).
- Thodoris Lykouris and Sergei Vassilvitskii. 2021. Competitive caching with machine learned advice. *Journal of the ACM (JACM)* 68, 4 (2021), 1–25.
- Ho-Yin Mak, Ying Rong, and Jiawei Zhang. 2015. Appointment scheduling with limited distributional information. *Management Science* 61, 2 (2015), 316–334.

-
- Microsoft. 2023. Bing AI. <https://www.bing.com/chat>.
- OpenAI. 2019. ChatGPT. <https://chat.openai.com>.
- OpenAI. 2023. GPT-4 technical report. arxiv 2303.08774. *View in Article* 2, 5 (2023).
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient generative LLM inference using phase splitting. *Power* 400, 700W (2023), 1–75.
- Perplexity. 2022. Perplexity AI. <https://www.perplexity.ai/>.
- Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2024. Efficient interactive LLM serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509* (2024).
- Malik Sallam. 2023. The utility of ChatGPT as an example of large language models in healthcare education, research and practice: Systematic review on the future perspectives and potential limitations. *MedRxiv* (2023), 2023–02.
- Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. 2024. Don’t Stop Me Now: Embedding Based Scheduling for LLMs. *arXiv preprint arXiv:2410.01035* (2024).
- Meixuan Wang, Yinyu Ye, and Zijie Zhou. 2025. LLM Serving Optimization with Variable Prefill and Decode Lengths. *arXiv preprint arXiv:2508.06133* (2025).
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022).
- Wenxun Xing and Jiawei Zhang. 2000. Parallel machine scheduling with splitting jobs. *Discrete Applied Mathematics* 103, 1-3 (2000), 259–269.
- Heng Yang, Yinyu Ye, and Jiawei Zhang. 2003. An approximation algorithm for scheduling two parallel machines with capacity constraints. *Discrete Applied Mathematics* 130, 3 (2003), 449–467.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- David W Zhang, Corrado Rainone, Markus Peschl, and Roberto Bondesan. 2023. Robust scheduling with gflownets. *arXiv preprint arXiv:2302.05446* (2023).
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric Xing, et al. 2023a. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. *arXiv preprint arXiv:2309.11998* (2023).
- Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023b. Response length perception and sequence scheduling: An llm-empowered llm inference pipeline. *Advances in Neural Information Processing Systems* 36 (2023), 65517–65530.

- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670* (2024).
- Minglong Zhou, Melvyn Sim, and Shao-Wei Lam. 2022. Advance admission scheduling via resource satisficing. *Production and Operations Management* 31, 11 (2022), 4002–4020.

Proofs of Statements

EC.1. Memory-Preserving Proof Technique for Theorem 1

In this section, we study the relationship between the total end-to-end latency and the memory constraint M . Although it is generally intractable to derive a closed-form expression for total latency as a function of M , we can still make meaningful comparisons between algorithms that satisfy certain structural properties, which will be formally defined in Definitions EC.1 and EC.2. Specifically, by analyzing how total latency scales under different memory capacities for such algorithms, we can bound their relative performance. Based on this approach, we construct a benchmark algorithm and compare its total latency with that of $\mathcal{A}_{\max}$ to establish the competitive ratio.

To introduce the structural ideas underlying our analysis, we first define the notion of the *corresponding order* between two schedules in Definition EC.1. This concept formalizes when two schedules maintain the same relative processing order, even if their exact starting times differ.

DEFINITION EC.1 (CORRESPONDING ORDER). *Let $I = (I_0, I_1, \dots, I_T)$ and $I' = (I'_0, I'_1, \dots, I'_T)$ be two schedules produced by possibly different algorithms. We say that I and I' **preserve the corresponding order** if, for any two jobs $n_1 \in I_i \cap I'_{i'}$ and $n_2 \in I_j \cap I'_{j'}$, it holds that*

$$(i - j)(i' - j') \geq 0.$$

*In other words, if n_1 is scheduled earlier than n_2 in I , it is also scheduled no later than n_2 in I' , and vice versa. Furthermore, we say that I' is **delayed relative to** I , denoted $I \leq I'$, if for every job $n_0 \in I_i \cap I'_{i'}$, we have $i \leq i'$.*

Definition EC.1 focuses on the relative order of job processing rather than their exact starting times. This abstraction is crucial because, when comparing scheduling strategies (e.g., shortest-job-first or longest-job-first) under different memory capacities M , the exact batch compositions and starting times may vary significantly. Due to the linearly increasing memory usage during processing, tracking exact timings is highly complex. However, the key observation is that for scheduling strategies based purely on job ordering, the relative order of job processing remains stable even as M changes. For instance, shortest-job-first will always prioritize jobs according to their output lengths, independent of the specific value of M .

EXAMPLE EC.1. *Given the memory capacity $M = 7$ with $n = 4$, $s = 1$ and $\mathbf{o} = (1, 2, 3, 4)$. Consider the following different input schedules $I_1 = (\{1, 3\}, \emptyset, \{2\}, \{4\})$ and $I_2 = (\{3\}, \{1, 2\}, \emptyset, \{4\})$. We find*

that every request in I_1 scheduled earlier does not start processing later in I_2 and vice versa. This means that I_1 and I_2 preserve the corresponding order. However, the change of requests 1 and 2 shows that neither I_1 nor I_2 are delayed relative to the other. Moreover, let $I_3 = (\{1, 2, 3\}, \emptyset, \{4\})$ which is the optimal input sequence. We can see that I_1, I_2, I_3 preserve the corresponding order pairwise. In addition, because all jobs in I_3 begin earlier, both I_1 and I_2 are delayed relative to I_3 , $I_3 \leq I_1, I_2$.

Next, to meaningfully compare the total latency of two schedules, it is not sufficient for them to simply preserve the same corresponding order. We must also ensure that both schedules fully utilize the available memory—that is, they do not intentionally leave memory unused in a way that artificially increases latency. However, because the memory usage of each job grows linearly during processing, it is nontrivial to characterize what it means for a schedule to fully utilize memory. To formalize this notion, we introduce the concept of an *optimally packed* schedule in Definition EC.2.

DEFINITION EC.2 (OPTIMALLY PACKED SCHEDULE). *Let M denote the KV cache memory limit. A schedule $I = (I_0, I_1, \dots, I_T)$ is said to be **optimally packed** if for every job $n_0 \in I_i$ with $i \in [T]$, the following holds: Let*

$$j := \max\{k \in [0, i-1] : I_k \neq \emptyset\}$$

be the most recent time before i such that I_j is non-empty. Then, for any $k \in [j, i)$, moving n_0 to batch I_k would result in exceeding the memory limit M at some point during processing.

Definition EC.2 captures the idea that a schedule is memory-efficient in a myopic sense: a job cannot be moved earlier into any nearby batch—specifically, between its original batch and the nearest previous non-empty batch—without violating the memory constraint. The definition focuses only on local perturbations to ensure that no memory is intentionally left unused in a way that could have allowed earlier scheduling within immediate reach. This notion of optimal packing is thus well-suited for analyzing algorithms that follow natural, sequential batching processes. Combining Definitions EC.1 and EC.2, we are now prepared to compare the total latency of two schedules that (i) preserve the same corresponding order and (ii) are both optimally packed, but operate under different memory capacities.

PROPOSITION EC.1 (Latency Scaling under Reduced Memory). *Let I and I' be two schedules that preserve the corresponding order and are both optimally packed. Suppose I satisfies a memory constraint of M , while I' satisfies a memory constraint of βM , where $\beta \in (0, 1)$. Then, the total end-to-end latencies satisfy the following inequality:*

$$TEL(\mathbf{o}; I') \leq \beta^{-1} TEL(\mathbf{o}; I). \tag{EC.1}$$

Proof of Proposition EC.1 We aim to show that, under the given memory constraints, the total end-to-end latency of the input sequence I' is at most β^{-1} times that of the input sequence I .

First, fix the input order $\mathbf{o} = (o_1, o_2, \dots, o_n)$. Assume that both schedules I and I' are optimally packed, meaning that at every time step, the system processes as many requests as permitted by the memory constraint.

Since the memory capacity in I' is scaled by a factor $\beta \in (0, 1)$ relative to that in I , and each request o_i consumes the same amount of memory in both schedules, it follows that at each moment, the number of requests being processed in I' is β times the number in I .

Let L_i and L'_i denote the completion times of request i under schedules I and I' , respectively. Our goal is to relate L'_i to L_i for each $i \in [n]$.

Because I' has a memory capacity reduced by a factor β , and because both schedules are fully packed, the overall processing rate in I' is slowed down by a factor of β compared to I . Thus, to complete the same workload, the time needed under I' is multiplied by a factor of β^{-1} . This implies that the completion time of each request satisfies

$$L'_i \leq \beta^{-1} L_i, \quad \forall i \in [n]. \quad (\text{EC.2})$$

Now, recall that the total end-to-end latency TEL for an input sequence is defined as the sum of the completion times for all requests:

$$\text{TEL}(\mathbf{o}; I) = \sum_{i=1}^n L_i, \quad \text{TEL}(\mathbf{o}; I') = \sum_{i=1}^n L'_i. \quad (\text{EC.3})$$

Substituting the earlier established relation between L_i and L'_i , we have:

$$\text{TEL}(\mathbf{o}; I') = \sum_{i=1}^n L'_i \leq \sum_{i=1}^n \beta^{-1} L_i = \beta^{-1} \text{TEL}(\mathbf{o}; I). \quad (\text{EC.4})$$

Thus, we conclude that

$$\text{TEL}(\mathbf{o}; I') \leq \beta^{-1} \text{TEL}(\mathbf{o}; I).$$

□

To apply Proposition EC.1 in our analysis, we first need to construct an auxiliary algorithm that (i) preserves the same corresponding order as $\mathcal{A}_{\max}$ and (ii) has a competitive ratio that is easier to analyze. By comparing $\mathcal{A}_{\max}$ with this auxiliary algorithm, we can bound the performance of $\mathcal{A}_{\max}$. Noting that $\mathcal{A}_{\max}$ treats all jobs as identical and selects a batch by randomly sampling the maximum feasible number of requests, we construct a fully randomized benchmark algorithm, $\mathcal{A}_{\text{random}}$ designed specifically for this comparison. The details of $\mathcal{A}_{\text{random}}$ can be found in Algorithm 4 in Appendix EC.3.

Algorithm $\mathcal{A}_{\text{random}}$ is an auxiliary algorithm constructed for analysis purposes, and it is not intended to be practically implementable. In particular, it requires full knowledge of all true output lengths $\mathbf{o} = (o_1, o_2, \dots, o_n)$ as input, similar to a hindsight algorithm that perfectly predicts the realized output lengths of all jobs. More specifically, Algorithm $\mathcal{A}_{\text{random}}$ first randomly samples a permutation of all n requests and then processes the jobs according to this fixed order. At each step, it sequentially adds requests following the permutation, ensuring that the memory constraint in Equation (3) is satisfied at all times. Since it has access to the true values of $\mathbf{o}$, the algorithm can achieve an optimally packed schedule. Next, we present a theorem which characterizes the competitive ratio of Algorithm $\mathcal{A}_{\text{random}}$ and states that for any scheduling policy, the worst case always happens when the true output length o_i is either ℓ or u for all $i \in [n]$.

THEOREM EC.1 (Worst-Case Realizations and Competitive Ratio of $\mathcal{A}_{\text{random}}$). *For any feasible scheduling policy π , the worst-case competitive ratio is attained when each output length o_i takes either the lower bound ℓ or the upper bound u . That is,*

$$CR(\pi) := \sup_{\mathbf{o} \in [\ell, u]^n} \frac{\mathbb{E}[TEL(\mathbf{o}; \pi)]}{TEL(\mathbf{o}; H-SF)} = \sup_{\mathbf{o} \in \{\ell, u\}^n} \frac{\mathbb{E}[TEL(\mathbf{o}; \pi)]}{TEL(\mathbf{o}; H-SF)}. \quad (\text{EC.5})$$

Moreover, the competitive ratio of the auxiliary algorithm $\mathcal{A}_{\text{random}}$ satisfies

$$CR(\mathcal{A}_{\text{random}}) \leq \frac{1 + \alpha^{-1}}{2} + \mathcal{O}\left(\frac{1}{M}\right), \quad (\text{EC.6})$$

as $M \rightarrow +\infty$, where $\alpha = \ell/u$.

The proof of Theorem EC.1 can be found in Appendix EC.3. Next, we compare two versions of Algorithm $\mathcal{A}_{\text{random}}$:

- $\mathcal{A}_{\text{random}}(M)$: Algorithm $\mathcal{A}_{\text{random}}$ run with memory capacity M ,
- $\mathcal{A}_{\text{random}}(\alpha M)$: Algorithm $\mathcal{A}_{\text{random}}$ run with memory capacity αM , where $\alpha = \ell/u$.

Both versions produce randomized schedules based on random permutations of the input requests. Recall that Proposition EC.1 states that for any pair of schedules that preserve the corresponding order and are optimally packed, we can directly compare their total latencies. To formalize the connection between $\mathcal{A}_{\text{random}}(M)$ and $\mathcal{A}_{\text{random}}(\alpha M)$, we introduce the following probabilistic coupling:

DEFINITION EC.3 (PERMUTATION COUPLING). *Sample a random permutation σ of $[n]$ uniformly at random.*

- Under $\mathcal{A}_{\text{random}}(M)$, schedule the jobs according to σ , subject to memory capacity M .

- Under $\mathcal{A}_{\text{random}}(\alpha M)$, schedule the jobs according to the same permutation σ , subject to memory capacity αM .

By this probabilistic coupling, each random schedule generated by $\mathcal{A}_{\text{random}}(M)$ is paired with a corresponding schedule generated by $\mathcal{A}_{\text{random}}(\alpha M)$, and each pair occurs with the same probability. Now, since the coupled schedules preserve the same corresponding order and are optimally packed, Proposition EC.1 implies that the total latency of each realization under $\mathcal{A}_{\text{random}}(\alpha M)$ is at most α^{-1} times the corresponding latency under $\mathcal{A}_{\text{random}}(M)$. Taking expectations over the coupling, we conclude that the expected total latency under $\mathcal{A}_{\text{random}}(\alpha M)$ is at most α^{-1} times the expected total latency under $\mathcal{A}_{\text{random}}(M)$, namely,

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(\alpha M))] \leq \alpha^{-1} \mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(M))]. \quad (\text{EC.7})$$

As Theorem EC.1 establishes the competitive ratio of $\mathcal{A}_{\text{random}}(M)$, to bound the competitive ratio of $\mathcal{A}_{\text{max}}$, it suffices to compare the expected total latency between $\mathcal{A}_{\text{max}}$ and $\mathcal{A}_{\text{random}}(M)$. By Equation (EC.7), we can introduce $\mathcal{A}_{\text{random}}(\alpha M)$ as an intermediate benchmark. The next lemma shows that it is easier to compare the expected total latency between $\mathcal{A}_{\text{max}}$ with memory capacity M and $\mathcal{A}_{\text{random}}(\alpha M)$.

LEMMA EC.1. *Denote $\mathcal{A}_{\text{random}}(\alpha M)$ as Algorithm $\mathcal{A}_{\text{random}}$ run with memory capacity αM , where $\alpha = \frac{\ell}{u}$, then we have*

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{max}})] \leq \mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(\alpha M))]. \quad (\text{EC.8})$$

The proof of Lemma EC.1 can be found in Appendix EC.3. Combining the result in Lemma EC.1 and Equation (EC.8), we are ready to give the competitive ratio upper bound to $\mathcal{A}_{\text{max}}$, which states in Theorem 1. The proof can also be found in Appendix EC.3.

EC.2. Supplementary Materials for Section 3

EC.3. Supplementary Materials for Section EC.1

Proof of Theorem EC.1 We split the proof into two parts, where in part 1, we show that the worst case happens only if the output length of each o_i is either ℓ or u . In part 2, we prove for the upper bound of the competitive ratio of $\mathcal{A}_{\text{random}}$.

—Part 1—

Without loss of generality, assume the output lengths are ordered as $o_1 \leq o_2 \leq \dots \leq o_n$. We analyze how the total end-to-end latency changes when the output length of a single request is increased or decreased by one unit.

Algorithm 3: H-SF

Input: Memory Capacity M , prompt requests $\mathbf{o} = (o_1, o_2, \dots, o_n)$.**Output:** Processing Sequence: $I(I_0, I_1, \dots, I_T)$.**while** *there exist waiting requests* **do**

Let R_t be the set of waiting prompts at time t . Let S_t be the set of tokens currently processing at time t .

Find the set $I_t \subset R_t$ with largest cardinality such that Equation (1) hold for all $t' \geq t$.

Process the requests in $I_t \cup S_t$ and update $R_{t+1} = R_t \setminus I_t$.

end

First, denote π as the scheduling policy $\mathcal{A}_{\text{random}}$ and fix an input vector $\mathbf{o} = (o_1, \dots, o_n)$. Consider decreasing the output length of job i from o_i to $o_i - 1$, producing a new instance $\mathbf{o}^-$. Because the job order remains unchanged, the start time of job i is unaffected. The main effect is that some requests scheduled after job i may now complete earlier, potentially reducing their latency by one unit.

Let F denote the set of jobs whose completion times are advanced by one unit due to this reduction. Then, the total latency under $\mathbf{o}^-$ satisfies

$$\mathbb{E}[\text{TEL}(\mathbf{o}^-; \pi)] = \mathbb{E}[\text{TEL}(\mathbf{o}; \pi)] - 1 - |F|.$$

Similarly, consider increasing o_i to $o_i + 1$, producing a new instance $\mathbf{o}^+$. Let B denote the set of jobs whose completion times are delayed by one unit. Then,

$$\mathbb{E}[\text{TEL}(\mathbf{o}^+; \pi)] = \mathbb{E}[\text{TEL}(\mathbf{o}; \pi)] + 1 + |B|.$$

Since the scheduling policy π does not know the true output lengths, and because jobs with the same output size are symmetric under random permutations, we can pair the effects of decreasing and increasing a job's output length. Taking expectations yields:

$$\mathbb{E}[\text{TEL}(\mathbf{o}^-; \pi)] + \mathbb{E}[\text{TEL}(\mathbf{o}^+; \pi)] = 2\mathbb{E}[\text{TEL}(\mathbf{o}; \pi)].$$

Now consider the hindsight shortest-job-do-first's latency $\text{TEL}(\mathbf{o}; \text{H-SF})$ for each instance. Since the hindsight policy knows all output lengths in advance, changing the output length of a single job affects only the scheduling of subsequent jobs. Moreover, because jobs with smaller output lengths consume less memory and are generally processed earlier under H-SF, reducing a job's output length tends to bring forward more jobs than increasing it delays. This gives

$$\text{TEL}(\mathbf{o}^-; \text{H-SF}) + \text{TEL}(\mathbf{o}^+; \text{H-SF}) \leq 2\text{TEL}(\mathbf{o}; \text{H-SF}).$$

Using these observations, we can now compare the competitive ratios:

$$\frac{\mathbb{E}[\text{TEL}(\mathbf{o}; \pi)]}{\text{TEL}(\mathbf{o}; \text{H-SF})} \leq \max \left\{ \frac{\mathbb{E}[\text{TEL}(\mathbf{o}^-; \pi)]}{\text{TEL}(\mathbf{o}^-; \text{H-SF})}, \frac{\mathbb{E}[\text{TEL}(\mathbf{o}^+; \pi)]}{\text{TEL}(\mathbf{o}^+; \text{H-SF})} \right\}.$$

This inequality shows that either decreasing or increasing the output length of a single job by one unit leads to a higher or equal competitive ratio.

Finally, suppose there exists an instance $\mathbf{o} \in [\ell, u]^n$ achieving the supremum in the definition of the competitive ratio. If there exists a job i such that $\ell < o_i < u$, then by perturbing o_i to $o_i - 1$ or $o_i + 1$ (and reordering the jobs if necessary), we can produce an instance with a higher or equal competitive ratio. Repeating this process finitely many times eventually produces an instance where each $o_i \in \{\ell, u\}$.

Therefore,

$$\sup_{\mathbf{o} \in [\ell, u]^n} \frac{\mathbb{E}[\text{TEL}(\mathbf{o}; \pi)]}{\text{TEL}(\mathbf{o}; \text{H-SF})} = \sup_{\mathbf{o} \in \{\ell, u\}^n} \frac{\mathbb{E}[\text{TEL}(\mathbf{o}; \pi)]}{\text{TEL}(\mathbf{o}; \text{H-SF})},$$

completing the proof.

—Part 2—

By Part 1, suppose there are N_ℓ requests with output length ℓ and N_u requests with output length u , where N_ℓ and N_u are large. By definition, the expected total latency under Algorithm $\mathcal{A}_{\text{random}}$ can be decomposed as

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}})] = N_\ell \mathbb{E}[L_i \mid o_i = \ell] + N_u \mathbb{E}[L_j \mid o_j = u],$$

where L_i and L_j denote the end-to-end latencies of requests with output lengths ℓ and u , respectively.

Since Algorithm $\mathcal{A}_{\text{random}}$ generates a random permutation and processes requests sequentially while ensuring optimal packing, the schedule is optimally packed by construction. The total amount of work, measured in terms of memory consumption over time, is upper bounded by

$$\ell \lceil \frac{N_\ell}{\lfloor \frac{M}{s+\ell} \rfloor} \rceil + u \lceil \frac{N_u}{\lfloor \frac{M}{s+u} \rfloor} \rceil \leq \left(1 + \mathcal{O}\left(\frac{1}{M}\right)\right) \left(\frac{N_\ell}{M} \ell(s+\ell) + \frac{N_u}{M} u(s+u)\right),$$

where M is the memory capacity, and $s + \ell$ and $s + u$ represent the peak memory footprint for requests of type ℓ and u , respectively.

Due to the symmetry of random sampling, each request of a given type has the same expected latency. Furthermore, the expected latency for each request is at most half of the total processing time for all requests, because in expectation, a request will be located near the middle of the cumulative schedule. Thus, we have

$$\mathbb{E}[L_i \mid o_i = \ell], \mathbb{E}[L_j \mid o_j = u] \leq \frac{1}{2} \left(1 + \mathcal{O}\left(\frac{1}{M}\right)\right) \left(\frac{N_\ell}{M} \ell(s+\ell) + \frac{N_u}{M} u(s+u)\right).$$

Next, consider the hindsight shortest-job-do-first's total latency $\text{TEL}(\mathbf{o}; \mathbf{H}\text{-SF})$. Since the offline scheduler knows all output lengths in advance and it can prioritize smaller jobs first, requests with output length l have latency at least $N_\ell \frac{1}{2} \ell \lfloor \frac{N_\ell}{s+\ell} \rfloor$. Also, requests with output length u are subject to a delay of $\ell \lfloor \frac{N_\ell}{s+\ell} \rfloor$ periods, and their latency is more than $N_\ell \left(\ell \lfloor \frac{N_\ell}{s+\ell} \rfloor + \frac{1}{2} u \lfloor \frac{N_u}{s+u} \rfloor \right)$. The corresponding total latency is at least

$$\text{TEL}(\mathbf{o}; \mathbf{H}\text{-SF}) \geq \frac{1}{2} \left(1 + \mathcal{O}\left(\frac{1}{M}\right) \right)^{-1} \left(l(s+\ell) \frac{N_\ell^2}{M} + 2l(s+\ell) \frac{N_\ell N_u}{M} + u(s+u) \frac{N_u^2}{M} \right).$$

Taking the ratio, we obtain

$$\frac{\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}})]}{\text{TEL}(\mathbf{o}; \mathbf{H}\text{-SF})} \leq \left(1 + \mathcal{O}\left(\frac{1}{M}\right) \right) \frac{(N_\ell + N_u)(N_\ell \ell(s+\ell) + N_u u(s+u))}{l(s+\ell)N_\ell^2 + 2l(s+\ell)N_\ell N_u + u(s+u)N_u^2}.$$

Since $\alpha = \ell/u$, simplifying the expression yields

$$\frac{\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}})]}{\text{TEL}(\mathbf{o}; \mathbf{H}\text{-SF})} \leq \left(1 + \mathcal{O}\left(\frac{1}{M}\right) \right) \left(1 + \frac{1 - \alpha^2}{2(\alpha + \alpha^2)} \right) = \frac{1 + \alpha^{-1}}{2} + \mathcal{O}\left(\frac{1}{M}\right).$$

Thus, the competitive ratio of $\mathcal{A}_{\text{random}}$ is at most $\frac{1 + \alpha^{-1}}{2} + \mathcal{O}(\frac{1}{M})$, as claimed.

□

Proof of Lemma EC.1 We first define a probabilistic coupling between $\mathcal{A}_{\text{max}}$ and $\mathcal{A}_{\text{random}}(\alpha M)$:

DEFINITION EC.4 (PERMUTATION COUPLING FOR $\mathcal{A}_{\text{max}}$ AND $\mathcal{A}_{\text{RANDOM}}(\alpha M)$). *Sample a random permutation σ of $[n]$ uniformly at random.*

- Under $\mathcal{A}_{\text{max}}$, schedule the jobs according to σ , treating all output lengths as u and applying memory constraint M .
- Under $\mathcal{A}_{\text{random}}(\alpha M)$, schedule the jobs according to the same permutation σ , using the true output lengths and memory constraint αM .

By construction, under this coupling, the two schedules share the same processing order.

Denote by $I_{\mathcal{A}_{\text{max}}}$ the input sequence generated by $\mathcal{A}_{\text{max}}$ with memory M , and $I_{\mathcal{A}_{\text{random}}(\alpha M)}$ the input sequence generated by $\mathcal{A}_{\text{random}}(\alpha M)$ with memory αM . We claim that, under the coupling,

$$I_{\mathcal{A}_{\text{max}}} \leq I_{\mathcal{A}_{\text{random}}(\alpha M)},$$

meaning that every request in $\mathcal{A}_{\text{max}}$ is delayed to $\mathcal{A}_{\text{random}}(\alpha M)$. The definition of one schedule is delayed to the other is defined in Definition EC.1.

The reasoning is as follows. At any time, if there remains sufficient memory (at least $s+u$) under $\mathcal{A}_{\text{max}}$, the algorithm immediately initiates a new request, treating it as requiring output length u .

Since u is the worst-case (largest) output length and $\mathcal{A}_{\max}$ batches jobs based on this conservative estimate, the memory utilization rate per job under $\mathcal{A}_{\max}$ is at least $\frac{s+\ell}{s+u} \geq \alpha$.

In contrast, under $\mathcal{A}_{\text{random}}(\alpha M)$, the algorithm uses the true output lengths, which can be as small as ℓ , and thus may utilize the memory less aggressively. Therefore, $\mathcal{A}_{\max}$ tends to process more requests earlier compared to $\mathcal{A}_{\text{random}}(\alpha M)$. As a result, for every request, the number of preceding completed jobs is at least as large under $\mathcal{A}_{\max}$ as under $\mathcal{A}_{\text{random}}(\alpha M)$, leading to earlier or equal start times.

Consequently, we have

$$\text{TEL}(\mathbf{o}; I_{\mathcal{A}_{\max}}) \leq \text{TEL}(\mathbf{o}; I_{\mathcal{A}_{\text{random}}(\alpha M)})$$

for each realization.

Taking expectations over the randomness in the permutation coupling, we conclude

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\max})] \leq \mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(\alpha M))],$$

which completes the proof. $\square$

Proof of Theorem 1 By Lemma EC.1, we have

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\max})] \leq \mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(\alpha M))]. \quad (\text{EC.9})$$

In addition, we define a probabilistic coupling between $\mathcal{A}_{\text{random}}(M)$ and $\mathcal{A}_{\text{random}}(\alpha M)$.

DEFINITION EC.5 (PERMUTATION COUPLING FOR $\mathcal{A}_{\text{RANDOM}}(M)$ AND $\mathcal{A}_{\text{RANDOM}}(\alpha M)$). *Sample a random permutation σ of $[n]$ uniformly at random.*

- Under $\mathcal{A}_{\text{random}}(M)$, schedule the jobs according to the same permutation σ , using the true output lengths and memory constraint M .
- Under $\mathcal{A}_{\text{random}}(\alpha M)$, schedule the jobs according to the same permutation σ , using the true output lengths and memory constraint αM .

Denote by $I_{\mathcal{A}_{\text{random}}(M)}$ the input sequence generated by $\mathcal{A}_{\text{random}}(M)$ with memory M , and $I_{\mathcal{A}_{\text{random}}(\alpha M)}$ the input sequence generated by $\mathcal{A}_{\text{random}}(\alpha M)$ with memory αM . By construction, we see that they preserve the corresponding order and are both optimally packed. Recall Proposition EC.1, we have

$$\text{TEL}(\mathbf{o}; I_{\mathcal{A}_{\text{random}}(M)}) \leq \alpha^{-1} \text{TEL}(\mathbf{o}; I_{\mathcal{A}_{\text{random}}(\alpha M)}). \quad (\text{EC.10})$$

Taking expectations over the randomness in the permutation coupling, we arrive at

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(\alpha M))] \leq \alpha^{-1} \mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(M))].$$

Combining two results, we have

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\max})] \leq \alpha^{-1} \mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\text{random}}(M))].$$

Taking the competitive ratio, we obtain

$$\text{CR}(\mathcal{A}_{\max}) \leq \alpha^{-1} \text{CR}(\mathcal{A}_{\text{random}}).$$

According to Theorem EC.1, the competitive ratio of the algorithm $\mathcal{A}_{\max}$ satisfies

$$\text{CR}(\mathcal{A}_{\max}) \leq \alpha^{-1} \text{CR}(\mathcal{A}_{\text{random}}) \leq \frac{\alpha^{-1}(1 + \alpha^{-1})}{2} + \mathcal{O}\left(\frac{1}{M}\right), \quad (\text{EC.11})$$

as $M \rightarrow +\infty$, which is the desired conclusion. $\square$

Algorithm 4: Auxiliary $\mathcal{A}_{\text{random}}$

Input: Memory Capacity M , prompt size s , output length $\mathbf{o} = (o_1, o_2, \dots, o_n)$.

Output: Processing sequence $I = (I_0, I_1, \dots, I_T)$.

Randomly generate a permutation $(i_1, \dots, i_n)$ of $[n]$.

while *there exist waiting requests* **do**

Let R_t be the set of waiting prompts at time t . Let S_t be the set of tokens currently processing at time t .

Following the permutation order, successively add requests from R_t into the memory until no further addition is possible without violating Equation (3) at any $t' \geq t$.

Let $I_t \subset R_t$ be the selected set of prompts.

Process the requests in $I_t \cup S_t$ and update $R_{t+1} = R_t \setminus I_t$.

end

EC.4. Supplementary Materials for Section 4

Proof of Theorem 3 We analyze the computational complexity of Algorithm $\mathcal{A}_{\min}$ step by step.

First, at each time step, to select a batch of new requests, the algorithm sorts the available prompts based on the auxiliary values $\tilde{o}_i$. Since the number of concurrent prompts is at most proportional to the memory capacity M (specifically, at most M/s), sorting requires at most $\mathcal{O}(M \log M)$ operations.

Second, when verifying whether adding a request would violate the memory constraint, the algorithm must compute the projected memory usage at the next time step. Simulating the projected memory usage across all currently active requests requires $\mathcal{O}(M)$ time, since the number of active requests is again at most proportional to M .

Third, if a memory overflow is predicted, the algorithm must remove requests to bring memory usage back within the constraint. Since requests are already maintained in sorted order by $\tilde{o}_i$, selecting and removing jobs can be performed efficiently. The removal operation involves traversing the sorted list and updating memory usage estimates, which takes at most $\mathcal{O}(M)$ time overall.

Finally, updates to the auxiliary variables $\tilde{o}_i$ occur when output tokens are generated. These updates involve only local modifications and require $\mathcal{O}(1)$ time per token, for a total of at most $\mathcal{O}(M)$ operations per time step.

Combining all steps, the dominant operation per time step is the initial sorting, leading to an overall computational complexity of $\mathcal{O}(M \log M)$.

□

Proof of Theorem 4 We proceed by contradiction. Suppose there exists a feasible policy π that achieves a strictly smaller competitive ratio than $\mathcal{A}_{\min}$. Then, there must exist a time t at which π and $\mathcal{A}_{\min}$ make different decisions for the first time.

We first consider the case in which they choose to add different requests. By definition, $\mathcal{A}_{\min}$ always adds the request with the smallest pseudo-output length $\tilde{o}$. Suppose π chooses to start request i , while $\mathcal{A}_{\min}$ starts request j , with $\tilde{o}_i > \tilde{o}_j$. Then, from the property of conditional expectation, we have

$$\mathbb{E}[o_i \mid o_i \geq \tilde{o}_i] > \mathbb{E}[o_j \mid o_j \geq \tilde{o}_j].$$

Let the completion time of request j under policy π be t' . If we modify the schedule so that request j is started at time t and request i is delayed to complete at time t' , then the expected net change in latency is

$$\mathbb{E}[o_i \mid o_i \geq \tilde{o}_i] - \mathbb{E}[o_j \mid o_j \geq \tilde{o}_j],$$

which is strictly positive, implying that such a switch would reduce total expected latency.

Next, consider the case in which they choose to delete different requests at time t . Again, recall that $\mathcal{A}_{\min}$ always deletes the request with the smallest $\tilde{o}$. Suppose π deletes request i , and $\mathcal{A}_{\min}$ deletes request j , with $\tilde{o}_i > \tilde{o}_j$. This implies that request i has been processed for a longer time than request j . In this case, we observe that

$$\mathbb{E}[o_i \mid o_i \geq \tilde{o}_i] - \tilde{o}_i \leq \mathbb{E}[o_j \mid o_j \geq \tilde{o}_j] - \tilde{o}_j.$$

Moreover, the deleted request must be restarted, resulting in increased latency. The expected latency difference between the two policies is given by

$$((\mathbb{E}[o_j \mid o_j \geq \tilde{o}_j] - \tilde{o}_j) + \mathbb{E}[o_i \mid o_i \geq \tilde{o}_i]) - ((\mathbb{E}[o_i \mid o_i \geq \tilde{o}_i] - \tilde{o}_i) + \mathbb{E}[o_j \mid o_j \geq \tilde{o}_j]) \geq 0,$$

which simplifies to $\tilde{o}_i - \tilde{o}_j > 0$. Hence, switching the deletion from i to j would again yield a lower expected latency.

Finally, we exclude the case in which only one of the two algorithms performs an action at time t , while the other remains idle. By construction, $\mathcal{A}_{\min}$ always acts whenever an insertion or deletion is feasible, since its objective is to exploit as much useful information as possible under capacity constraints. Hence, if policy π attempts an insertion earlier than $\mathcal{A}_{\min}$, the action cannot increase information gain: the inserted request will either need to be deleted prematurely or will consume memory without benefit. If, on the other hand, π inserts later than $\mathcal{A}_{\min}$, then idle slots arise and the system is no longer optimally packed. Similarly, if π deletes a request earlier than $\mathcal{A}_{\min}$, then the partially processed work is wasted and no additional information is gained; deleting later than $\mathcal{A}_{\min}$ exceeds the memory budget, which necessarily deteriorates performance.

Combining both cases, we reach a contradiction to the assumption that π outperforms $\mathcal{A}_{\min}$. Therefore, we conclude that

$$\mathbb{E}[\text{TEL}(\mathbf{o}; \mathcal{A}_{\min})] \leq \mathbb{E}[\text{TEL}(\mathbf{o}; \pi)].$$

Taking the ratio of both sides, it follows that

$$\text{CR}(\mathcal{A}_{\min}) \leq \text{CR}(\pi).$$

□

Proof of Lemma 1 Since in period τ_i all requests are treated as having a lower bound $\tilde{o} = i$, the symmetry among inputs with true length $o = j$ implies that only a fraction $\frac{s+i}{s+j}$ can be completed without deletion. That is,

$$\mathbb{E}[c_{ij}|b_{ij}] = \frac{s+i}{s+j} b_{ij}.$$

Similarly, for deletions, we obtain the following,

$$\mathbb{E}[d_{ij}|b_{ij}] = \left(1 - \frac{s+i}{s+j}\right) b_{ij}.$$

□

Proof of Proposition 1 We proceed by induction. In the initial period τ_ℓ , all requests are processed directly, and the result follows from Lemma 1. In the next period $\tau_{\ell+1}$, all remaining requests with $o = \ell + 1$ must be completed, yielding $\mathbb{E}[c_{\ell+1, \ell+1}] = \mathbb{E}[d_{\ell, \ell+1}] = \frac{1}{s+\ell+1} x_{\ell+1} n$.

Since the deletion operation preserves the distribution, Equation 4 implies that

$$\mathbb{E}[c_{\ell+1, k}] = \frac{1}{s+\ell+1} \cdot \frac{s+\ell+1}{s+k} x_k n = \frac{1}{s+k} x_k n.$$

This suggests that in each period, $\mathbb{E}[c_{jj}] = \frac{1}{s+j} x_j n$, and the remaining requests with $o = j + 1$ satisfy $\mathbb{E}[c_{j+1, j+1}] = \frac{1}{s+j+1} x_{j+1} n$, which will be fully completed in the subsequent period. By the distribution preserving property, the result follows. □

Proof of Corollary 1 We first calculate the numerator:

$$\sum_{k=1}^u \left(\sum_{i=k}^u i x_i \right) \left(\sum_{i=k}^u \frac{1}{i} x_i + 2 \sum_{i=k+1}^u \frac{i-k}{i} x_i \right).$$

The coefficient of the cross term $x_i x_j$ (for $i \neq j$) is

$$\begin{aligned} & \frac{1}{ij} \sum_{k=1}^{\min(i,j)} (i^2(1+2j-2k) + j^2(1+2i-2k)) \\ &= \frac{1}{ij} (\min(i,j)(i^2+j^2) + 2\min(i,j)ij(i+j) - (i^2+j^2)\min(i,j)(\min(i,j)+1)) \\ &= \frac{\min(i,j)^2}{ij} ((i+j)^2 - 2\min(i,j)^2) \\ &= 2a_{ij}. \end{aligned}$$

In the case $i = j$, we have $i^2 x_i^2 = a_{ii} x_i^2$.

For the denominator, the coefficient of $x_i x_j$ for $i \neq j$ is

$$2\min(i,j)^2 = 2b_{ij},$$

and for $i = j$ we have $i^2 = b_{ii}$.

In conclusion, the competitive ratio can be written as a Rayleigh quotient,

$$\text{CR}(\mathcal{A}_{\min}) = \max_{\vec{x}} \frac{\vec{x}^\top \mathbf{A}_u \vec{x}}{\vec{x}^\top \mathbf{B}_u \vec{x}} + \mathcal{O}(\frac{1}{M}).$$

□

Proof of Lemma 3 We begin with $\mathbf{B}_u$. It is known that the matrix $\mathbf{M} = (\min(i,j))_{u \times u}$ is positive definite. Since $\mathbf{B}_u$ is the Hadamard square of M , i.e., $B = \mathbf{M} \circ \mathbf{M}$, and Hadamard powers of positive definite matrices preserve positive definiteness, we conclude that $\mathbf{B}_u \succ 0$.

To prove that $\mathbf{A}_u$ is also positive definite, we define a normalized matrix $\mathbf{A}'_u = (a'_{ij})_{u \times u}$,

$$a'_{ij} := \frac{2}{ij} \cdot a_{ij} - 1.$$

A direct calculation shows:

$$a'_{ij} = 2 \cdot \frac{\min(i,j)}{\max(i,j)} - \left(\frac{\min(i,j)}{\max(i,j)} \right)^2.$$

Let $r_{ij} := \frac{\min(i,j)}{\max(i,j)}$. Then:

$$a'_{ij} = 2r_{ij} - r_{ij}^2 = g(\log i - \log j),$$

where we use the identity

$$r_{ij} = \exp(-|\log i - \log j|),$$

and define

$$g(d) := 2e^{-|d|} - e^{-2|d|}.$$

We claim that $g(d)$ is a positive definite function on $\mathbb{R}$. Since g is even and continuous, by Bochner's theorem, it suffices to verify that its Fourier transform is nonnegative. Computing:

$$\widehat{g}(\xi) = \int_{-\infty}^{\infty} g(d)e^{-i\xi d} dd = 4 \cdot \frac{1}{1+\xi^2} - 4 \cdot \frac{1}{4+\xi^2} = \frac{12}{(1+\xi^2)(4+\xi^2)} > 0 \quad \forall \xi \in \mathbb{R}.$$

Hence g is positive definite.

By Schoenberg's theorem, for any finite set of real numbers $\{z_1, \dots, z_u\} \subset \mathbb{R}$, the matrix $(g(z_i - z_j))_{u \times u}$ is positive semi-definite. Moreover, if the points z_i are mutually distinct and $g(0) > 0$, then the matrix is strictly positive definite.

It follows that the matrix

$$\mathbf{A}'_u = (a'_{ij})_{u \times u} = (g(\log i - \log j))_{u \times u}$$

is strictly positive definite.

Since $\mathbf{A}_u = \frac{ij}{2}(a'_{ij} + 1)$, and the factor $\frac{ij}{2} > 0$, the transformation preserves positive definiteness. Therefore, $\mathbf{A}_u \succ 0$. $\square$

EC.5. Supplementary Materials for Section 5

Proof of Theorem 6 Let $\vec{x} = (x_\ell, 0, \dots, 0, x_u)$ with $0 < \ell < u$ and set $s = 0$. Since $x_k = 0$ unless $k \in \{\ell, u\}$, Theorem 2 yields

$$\begin{aligned} \text{num} &= \ell(\ell x_\ell + u x_u) \left(x_\ell + \frac{\ell}{u} x_u + 2 \frac{u-\ell}{u} x_u \right) + u^2 x_u \left(\frac{u-\ell}{u} \right)^2 x_u, \\ \text{den} &= \ell^2 x_\ell^2 + 2\ell^2 x_\ell x_u + u^2 x_u^2. \end{aligned}$$

Set $\alpha = \ell/u \in (0, 1)$ and $t = x_\ell/x_u$ (with $t \geq 0$). Dividing numerator and denominator yields:

$$\text{CR}(\mathcal{A}_{\min}) = 1 + \frac{\alpha(1-\alpha^2)t}{\alpha^2 t^2 + 2\alpha^2 t + 1} + \mathcal{O}\left(\frac{1}{M}\right).$$

Since $(\alpha t - 1)^2 \geq 0$ implies $\alpha^2 t^2 + 2\alpha^2 t + 1 \geq 2\alpha(1+\alpha)t$, we obtain:

$$\frac{\alpha(1-\alpha^2)t}{\alpha^2 t^2 + 2\alpha^2 t + 1} \leq \frac{1-\alpha}{2} \quad (\forall t \geq 0).$$

Therefore,

$$\text{CR}(\mathcal{A}_{\min}) \leq 1 + \frac{1-\alpha}{2} + \mathcal{O}\left(\frac{1}{M}\right) = \frac{3-\alpha}{2} + \mathcal{O}\left(\frac{1}{M}\right),$$

which completes the proof. $\square$

Algorithm 5: Revised $\mathcal{A}_{\min}$ under Multiple Prediction Intervals**Input:** Memory capacity M , prompt size s , output length lower bounds

$$\ell_j = \max\{\ell_k | \ell_k \leq o_i, k \in [m]\} \text{ for all } i \in [n].$$

Output: Processing sequence $I = (I_0, I_1, \dots, I_T)$.Initialize $\tilde{o}_i \leftarrow \ell_j$ for all $i \in [n]$.**while** *there exist unfinished requests* **do**

Let R_t be the set of waiting prompts at time t . Let S_t be the set of tokens currently processing at time t .

if *projected memory usage of S_t at time $t+1$ exceeds M* **then**

Sort S_t in ascending order of $\tilde{o}_i$; break ties uniformly at random.

Remove jobs one by one from S_t (following the sorted order) until projected memory usage at $t+1$ satisfies the memory constraint.

For each removed request i , update $\tilde{o}_i \leftarrow$ number of tokens already generated by request i .

end

Let S'_t be the set of remaining active requests after any removals.

Sort R_t in ascending order of $\tilde{o}_i$; break ties uniformly at random.

Select the largest subset $I_t \subset R_t$ (in order) such that adding I_t to S'_t satisfies the memory constraint in Equation (3).

Process the requests in $I_t \cup S'_t$.

Update $R_{t+1} = R_t \setminus I_t$.

end

Proof of Theorem 7 Under $\mathcal{A}_\ell$, the request of length ℓ is executed first and thus waits exactly ℓ batches; its average output token is generated at time $\frac{1}{2}\ell$. The request of length u is delayed by all ℓ tokens of the first request, in addition to its own u tokens, resulting in an average completion time of $\ell + u/2$. Multiplying these average sojourn times by the per-batch workload $(x_\ell + x_u)$ and then by the corresponding request sizes gives:

$$\text{TEL}(\mathbf{o}; \mathcal{A}_\ell) = \left[\ell\left(\frac{1}{2}\ell\right)(x_\ell + x_u)\right]x_\ell + \left[\left(\ell + \frac{1}{2}u\right)(x_\ell + x_u)\right]x_u = \frac{u^2}{2} \left[\alpha^2 x_\ell^2 + 3\alpha^2 x_\ell x_u + (2\alpha^2 + 1)x_u^2\right], \quad (1)$$

where $\alpha := \ell/u \in (0, 1)$.

In comparison, the Hindsight-Shortest First benchmark (**H-SF**) schedules every token as soon as capacity permits:

$$\text{TEL}(\mathbf{o}; \text{H-SF}) = \frac{u^2}{2} \left[\alpha^2 x_\ell^2 + 2\alpha^2 x_\ell x_u + x_u^2\right]. \quad (2)$$

Algorithm 6: $\mathcal{A}_\ell$

Input: Memory capacity M , prompt size s , output lengths $o_i \in \{\ell, u\}$ for all $i \in [n]$.

Output: Processing sequence $I = (I_0, I_1, \dots, I_T)$.

Initialize $\tilde{o}_i \leftarrow \ell$ for all $i \in [n]$.

while *there exist unfinished requests* **do**

Let R_t be the queue of waiting prompts at time t . Let S_t be the set of tokens currently processing.

For any $i \in S_t$ with more than ℓ tokens but not yet finished, remove i from S_t , set

$\tilde{o}_i \leftarrow u$, and move i to the end of R_t .

Let S'_t be the active set after updates.

Starting from the front of R_t , successively add requests to a set I_t as long as $I_t \cup S'_t$ satisfies the memory constraint, using $\tilde{o}_i$ for each $i \in I_t$.

Activate and process the requests in I_t ; update $R_{t+1} = R_t \setminus I_t$.

end

Taking the ratio gives:

$$\text{CR}(\mathcal{A}_\ell) = 1 + \frac{\alpha^2 x_\ell x_u + 2\alpha^2 x_u^2}{\alpha^2 x_\ell^2 + 2\alpha^2 x_\ell x_u + x_u^2} + \mathcal{O}\left(\frac{1}{M}\right). \quad (3)$$

To bound the fractional term, let $t := x_\ell/x_u \geq 0$ and define:

$$h(t) = \frac{\alpha^2 t + 2\alpha^2}{\alpha^2 t^2 + 2\alpha^2 t + 1}.$$

For $0 \leq \alpha \leq \frac{1}{2}$, the inequality $\alpha^2 t^2 + 2\alpha^2 t + 1 \geq 2\alpha(1 + \alpha)t$ implies $h(t) \leq \alpha/[2(1 + \alpha)]$ for all $t \geq 0$.

For $\frac{1}{2} \leq \alpha \leq 1$, $h(t)$ is maximized at $t = 0$, yielding $h(t) \leq 2\alpha^2$. Substituting these bounds produces the desired piecewise estimate:

$$\text{CR}(\mathcal{A}_\ell) \leq \begin{cases} 1 + \frac{\alpha}{2(1 + \alpha)}, & 0 \leq \alpha \leq \frac{1}{2}, \\ 1 + 2\alpha^2, & \frac{1}{2} \leq \alpha \leq 1, \end{cases} + \mathcal{O}\left(\frac{1}{M}\right),$$

completing the proof. $\square$

Proof of Theorem 8 We use the result of Theorem 2 to compute the denominator:

$$\sum_{k=1}^{\infty} k^3 q^{k-1} \left(2kq^{k-1} + \sum_{i=k+1}^{\infty} iq^{i-1} \right) = \frac{1 + 2q + 11q^2 + 8q^3 + 11q^4 + 2q^5 + q^6}{(1 - q)^6(1 + q)^4},$$

and the numerator:

$$\sum_{k=1}^{\infty} \left(\sum_{i=k}^{\infty} i^2 q^{i-1} \right) \left(\sum_{i=k}^{\infty} q^{i-1} + 2 \sum_{i=k+1}^{\infty} (i - k) q^{i-1} \right) = \frac{1 + 3q + 6q^2 + 3q^3 + q^4}{(1 - q)^6(1 + q)^2}.$$

Thus, we have

$$\text{CR}(\mathcal{A}_{\min}) = \frac{(1+q)^2(1+3q+6q^2+3q^3+q^4)}{1+2q+11q^2+8q^3+11q^4+2q^5+q^6} \leq \frac{14}{9} \approx 1.56.$$

□